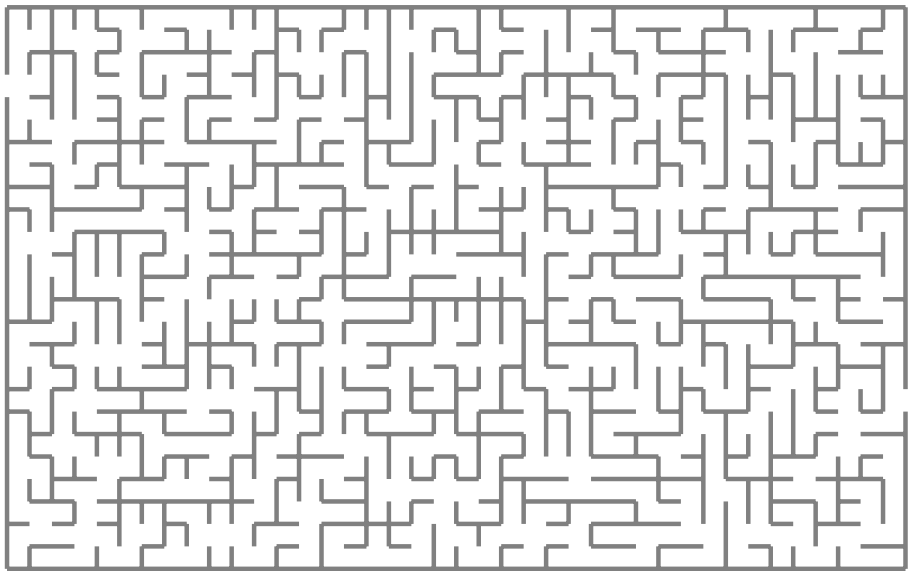
Software Engineering
Master Thesis Template

Version of 6th May 2026



Nick Glas

Software Engineering Master Thesis Template

THESIS

submitted in partial fulfilment of the
requirements for the degree of

MASTER OF SCIENCE

in

SOFTWARE ENGINEERING

by

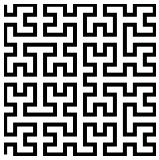
Nick Glas

under the supervision of

Dr. Ana Maria Oprescu (CCI, UvA)

May 2026

Master Software Engineering
Informatics Institute
FNWI, University of Amsterdam
Amsterdam, The Netherlands



Hilbert Institute for Space Research
Strekkerweg 41
1033 DA Amsterdam
The Netherlands

Thesis Committee

Examiner (chair):	Dr. Ana Maria Oprescu (CCI, UvA)
Second Reviewer:	Dr. Thomas L. van Binsbergen (CCI, UvA)
Daily Supervisor:	Dr. Damian Frölich (CCI, UvA)
External Supervisor:	Dennis Bijlsma (SIG)
Other members:	Prof. dr. ir. Cees Bakker (VU), Dr. Pierre de Vries (RUG)

Cover picture: A “random” maze.

Abstract

Neural network inference is increasingly deployed inside cloud-native software systems that must balance latency, deployability, scalability, and isolation. Splitting a model across microservices can improve modularity and make security boundaries more explicit, but it also forces intermediate activations to cross service interfaces. Each boundary adds serialization, transport, scheduling, and coordination cost. The central problem studied in this thesis is therefore not only where to split a neural network, but whether a decomposition that appears acceptable in a controlled local setting remains credible once it is moved into Kubernetes, Azure, and security-hardened deployment contexts.

This thesis addresses that problem through a staged experimental evaluation of ResNet-18 inference. A proof-of-concept gRPC-based microservice framework is used to compare monolithic inference, two-part model partitions, and synchronously chained multi-service deployments. The evaluation first screens coarse architectural split points under controlled local execution, then refines the selected region and explains the observed behaviour using activation-transfer size and compute distribution. The same decomposition family is then evaluated in local Kubernetes and Azure Kubernetes Service, after which the selected Azure deployment is hardened with service identity and mutual TLS. Confidential-execution mechanisms are treated as a later incremental protection layer on top of the secured Azure path.

The results show that monolithic execution remains the fastest baseline, but that not all microservice decompositions are equally costly. Early splits suffer from large activation transfers, while very late splits can become compute-degenerate. Mid-network boundaries provide the most credible two-part trade-off. Fine-grained refinement within this region does not materially improve latency, supporting the use of coarse architectural boundaries for later stages. In Kubernetes and Azure, latency increases with service count, but the increase is bounded and non-linear: later boundaries add less cost when the transferred activations are smaller. Azure preserves the condition ordering observed in local Kubernetes, indicating directional transfer despite different absolute latencies. Adding mutual TLS and service identity introduces measurable but bounded overhead, alongside clear operational complexity from sidecars, policies, and readiness delays.

Overall, the thesis concludes that microservice-based inference is viable only when

architectural partitioning, deployment context, and security techniques are evaluated together. The main engineering trade-off is not between monolithic and distributed inference in isolation, but between deployability and stronger isolation on one side and cumulative boundary-related overhead on the other.

Contents

List of Figures

List of Tables

Acknowledgements

This is were you can acknowledged people, if you prefer to do so.

Chapter 1

Introduction

1.1 Context

Deep neural network inference is increasingly embedded in software systems that must satisfy not only predictive requirements, but also operational ones such as latency, deployability, maintainability, scalability, and isolation. Edge-intelligence and split-computing surveys describe this shift as part of a broader move from purely centralized inference toward device, edge, and cloud collaboration [24, 37]. In many practical deployments, inference still runs as a monolithic component inside a single process. That architecture minimizes boundary-crossing overhead, but it also couples model execution into one deployment unit and limits architectural flexibility. A microservice-oriented design offers a different trade-off: the inference pipeline can be decomposed across service boundaries, allowing clearer separation of responsibilities, independent deployment, and more explicit control over resource placement.

For deep learning systems, however, decomposition is not only a software architecture decision. It is also a systems problem shaped by the structure of the neural network itself and by the environment in which the resulting services run. When a model is split across an execution boundary, intermediate activations must be serialized, transferred, and reconstructed before downstream computation can continue. Prior work on split computing and distributed DNN inference shows that these costs are highly sensitive to where the boundary is placed. DNNSplit demonstrates that split points should be selected systematically with respect to latency and cost rather than by arbitrary layer choice [19]. DeeperThings and survey work on DNN partitioning likewise show that early layers often produce large feature maps that are expensive to transmit, while later boundaries may reduce transfer volume but shift the compute balance too far toward one side of the partition [33, 35].

The deployment environment introduces an additional dimension to this problem. A split that is acceptable in a controlled local benchmark may behave differently once it is moved into containerized, orchestrated, or cloud-hosted execution, because container platforms, managed Kubernetes services, and cloud infrastructure add their own scheduling, networking, and virtualization effects [9, 10, 17]. Beyond performance, there is also a security dimension. As inference services move into shared or remote infrastruc-

ture, communication-level protections such as service identity and mutual TLS become relevant because they strengthen inter-service trust boundaries, while service-mesh studies show that this protection can add measurable runtime and resource cost [5, 38]. Confidential-computing techniques such as trusted execution environments may add a further protection layer, but their overhead and protection boundary are workload- and platform-dependent [11, 28]. This makes microservice-based inference a broader software engineering problem involving architectural partitioning, deployment infrastructure, and security mechanisms rather than only local latency optimisation.

1.2 Problem Statement

The central problem addressed in this thesis is how to design and evaluate deep neural network inference as a microservice system without losing control over performance. A poor split can make a distributed design clearly inferior to monolithic execution. Prior partitioning systems show that if the boundary is placed too early, the system may incur excessive latency because large activation tensors must cross the service interface; if the boundary is placed too late, communication cost decreases, but the downstream service may be left with too little computation to justify the split [16, 19, 33]. In other words, minimizing activation size alone is not enough; the chosen boundary must also preserve a meaningful compute distribution across the resulting services.

That architectural problem is only the first layer of the thesis. After candidate boundaries are identified, they must also be evaluated under progressively more realistic deployment conditions. Container orchestration and cloud deployment introduce additional scheduling, networking, and platform overhead that may alter the trade-offs seen in a local benchmark [6, 9]. Finally, if the selected deployment path is security-hardened, the system may incur further overhead in exchange for stronger inter-service authentication, encrypted communication, and, where applicable, confidential execution [36, 38]. The thesis therefore treats the problem as a staged evaluation pipeline: first identify suitable boundaries under controlled local conditions, then study how those candidates behave in Kubernetes and Azure deployments, and finally measure the impact of adding security hardening to the selected Azure deployment path.

1.3 Research Questions

This thesis is organised around two broader research strands. RQ1 studies how architectural split choice and deployment context affect microservice-based inference. RQ2 studies the performance and operational impact of security hardening on the selected Azure deployment path. The first hardening layer evaluates service identity and mutual TLS for inter-service traffic, while confidential-execution mechanisms such as trusted execution environments are treated as a later incremental protection layer.

The first stage is the coarse-boundary selection question:

> *RQ1.1: Which coarse architectural stage boundaries provide the most suitable two-part microservice decompositions of ResNet-18 under controlled local execu-*

tion, and how do their latency and boundary-crossing costs compare to monolithic inference?

RQ1.1 is intentionally framed as a coarse-boundary selection problem rather than an exhaustive search over every possible layer split. Its purpose is to identify architecturally meaningful two-part decompositions that can serve as defensible carry-forward candidates for later stages.

Subsequent RQ1 stages refine and extend this selection process. RQ1.2 narrows the analysis within the carried-forward architectural region to determine whether finer-grained split points reveal meaningful differences beyond the coarse stage level. Later RQ1 stages move the selected decompositions into broader deployment contexts, including Kubernetes-based orchestration in RQ1.4 and Azure-hosted execution in RQ1.5, so that the thesis can compare how the same architectural decision behaves under local, orchestrated, and cloud environments.

RQ2 then introduces a security-focused dimension to the thesis by hardening the selected deployment path and measuring the resulting performance and operational impact. In this way, the thesis moves from boundary selection, to deployment evaluation, to communication-level security and, where applicable, confidential-execution overhead.

1.4 Approach

The thesis uses a staged experimental methodology centred on ResNet-18, a residual architecture whose stage structure provides natural coarse split candidates [12]. The initial benchmark compares one monolithic baseline against four two-part split configurations placed after `layer1`, `layer2`, `layer3`, and `layer4`. These boundaries correspond to major architectural stages in the network and therefore provide a principled coarse sweep from early to late splits. This first stage is executed locally with CPU-only inference, FP32 precision, gRPC over localhost, fixed warmup and measurement windows, and functional parity checks between monolithic and split execution so that boundary effects can be isolated under controlled conditions, following systems-benchmarking guidance that emphasizes controlled comparisons and explicit reporting of measurement context [13].

Later stages deliberately relax that locality assumption. After coarse and fine-grained candidates have been selected, the same decomposition logic is carried into containerized and cloud-oriented settings to evaluate the effect of orchestration and hosted infrastructure. Finally, the selected deployment path is extended with communication-level security and, where applicable, confidential-execution mechanisms so that their overhead can be measured directly. This design follows the broader literature showing that DNN partitioning must be evaluated as a trade-off between communication and computation rather than as a purely architectural choice [20, 29, 31]. In this thesis, however, that trade-off is examined across multiple deployment layers rather than being treated as a purely local benchmarking problem.

1.5 Outline

The remainder of this thesis is organised as follows. Chapter 2 introduces the technical background required for understanding deep neural network inference, ResNet-style architectures, microservice-based deployment, and the broader systems context in which distributed inference is evaluated. Chapter 3 reviews prior literature on split computing, DNN partitioning, distributed inference, and the trade-offs that motivate this thesis. Chapter 4 describes the research methodology and the staged evaluation design used to move from local boundary screening to broader deployment and security contexts. Chapter 5 presents the empirical evaluation stages, beginning with RQ1.1 and continuing through the later deployment-oriented and security-hardening analyses. Chapter 6 interprets those findings in relation to the research questions and discusses their implications and limitations. Finally, Chapter 7 summarises the thesis and outlines directions for future work.

Chapter 2

Background

This chapter introduces the technical concepts needed to interpret the staged experiments. It focuses on three pieces of background: the ResNet-18 architecture used as the inference workload, the meaning of a split boundary in distributed DNN inference, and the Kubernetes-style service chain used in the later deployment stages. The more comparative discussion of prior work appears in chapter 3.

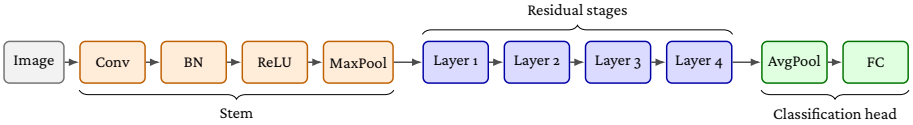


Figure 2.1: Top-level ResNet-18 inference pipeline used in this thesis. The model is treated as a stem, four residual stages, and a classification head.

ResNet-18 can be viewed as three major parts for the purposes of this thesis: an initial stem, four residual stages, and a classification head. The stem performs the first feature extraction and spatial downsampling through convolution, normalisation, activation, and max pooling. The four residual stages, denoted `layer1` through `layer4`, contain the main residual-block computation of the network. The classification head then reduces the final feature map through average pooling and applies the final fully connected classifier. This stage-based view follows the residual-network design introduced by He et al., where successive stages reduce spatial resolution while increasing channel depth [12].

This structure motivates the coarse split choices used later in RQ1.1. The thesis does not treat every primitive operation as an equally meaningful split candidate. Instead, the initial coarse sweep uses the four residual-stage boundaries after `layer1`, `layer2`, `layer3`, and `layer4`. These boundaries preserve the architectural stage structure of ResNet-18 and provide a controlled progression from early, high-activation regions to later, lower-activation regions. Splits inside the stem are excluded from the primary coarse sweep because they would place boundaries between tightly coupled primitive operations before substantial activation-size reduction. Splits inside the classification head are also excluded because the head contains only the final pooling and linear classi-

fication operations, leaving one side of the partition with negligible computation. This choice matches the broader partitioning literature, which treats split selection as a trade-off among activation-transfer cost, compute placement, and deployment context rather than as a search over depth alone [19, 33, 35].

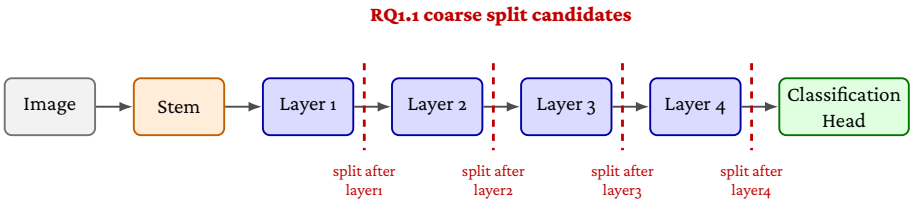


Figure 2.2: Coarse ResNet-18 partition boundaries evaluated in RQ1.1. The stem and classification head are collapsed into single blocks, while the four residual stages are shown as the candidate split locations.

As shown in fig. 2.2, RQ1.1 evaluates split points after the four major residual stages rather than inside the stem or classification head.

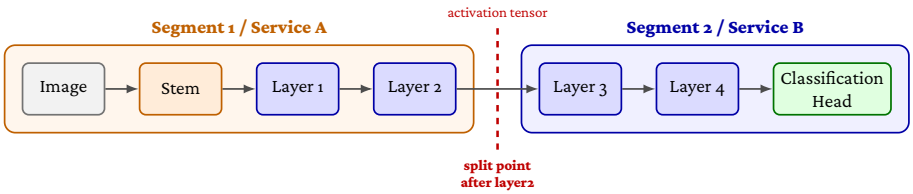


Figure 2.3: Example two-segment decomposition produced by a split after `layer2`. The first segment executes the stem through `layer2`; the second segment receives the intermediate activation tensor and executes `layer3` through the classification head.

A split boundary creates two execution segments, as illustrated for `split_after_layer2` in fig. 2.3. Prior split-inference systems use the same basic abstraction: an upstream segment computes an intermediate activation, transfers it to another execution environment, and the downstream segment completes the forward pass [7, 16].

The boundary-crossing interval used later in the thesis should therefore be read as a compound systems cost rather than as network transfer alone. It includes tensor serialization, RPC handling, transport through the service interface, downstream deserialization, remote computation after the boundary, and response handling. Recent microservice systems work shows that RPC paths can accumulate overhead through protocol processing, memory copies, socket operations, and network-stack traversal, even before the application-specific computation is considered [2, 39].

The block-level view matters because two boundaries with the same activation tensor can still assign different amounts of computation to the upstream and downstream services. This is why the thesis later evaluates a refined boundary inside `layer3` rather than treating activation size as the only explanatory variable. Prior work on fine-grained

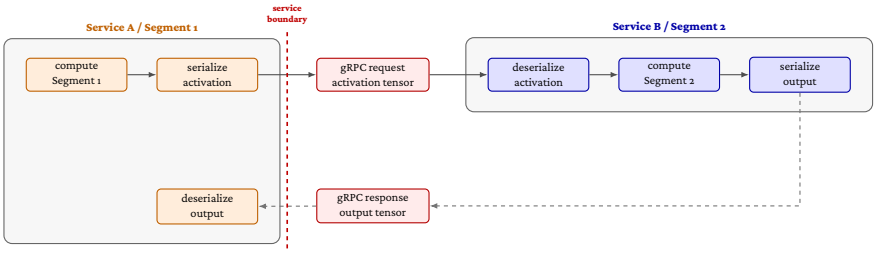


Figure 2.4: Runtime cost components introduced by a two-service inference boundary. In addition to the model computation on each side of the split, the intermediate activation must be serialized, transferred through gRPC, deserialized by the downstream service, and followed by response serialization and reconstruction. The boundary-crossing interval therefore includes serialization, RPC transport, remote execution, and response reconstruction.

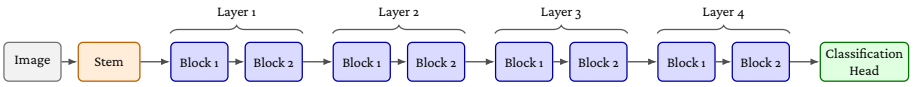


Figure 2.5: Simplified ResNet-18 block structure. The stem and classification head are shown as single components, while each residual stage is shown as two basic blocks.

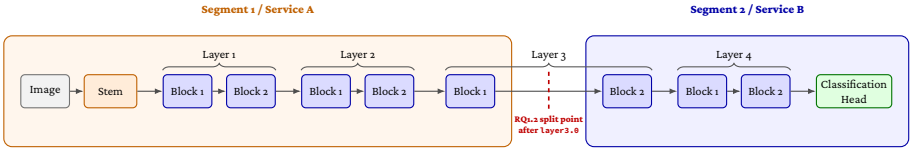


Figure 2.6: Block-level two-segment decomposition used in RQ1.2. The refined split point is placed inside `layer3`, after the first residual block. Segment 1 executes the stem through `layer3.0`; Segment 2 receives the intermediate activation tensor and executes `layer3.1` through the classification head.

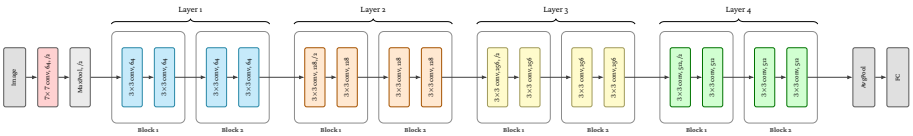


Figure 2.7: Detailed ResNet-18 structure with BasicBlock containers. Each residual stage contains two BasicBlocks, and each BasicBlock contains two convolution operations. The first block of layers 2–4 includes a projection shortcut for downsampling.

and hierarchical DNN partitioning similarly argues that useful split selection requires awareness of the model's internal computational structure [21, 31, 34].

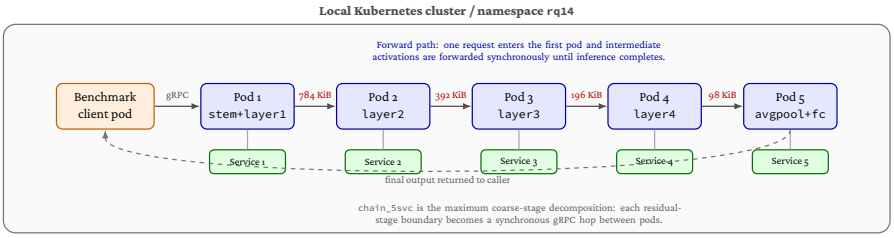


Figure 2.8: Kubernetes pod and service placement for the RQ1.4 five-service chain. The benchmark client and all inference pods are placed on the same Kubernetes node, while each model segment is exposed through a Kubernetes Service and communicates using synchronous gRPC forwarding. The labels above the arrows show the activation-transfer size at each coarse ResNet-18 boundary.

In the Kubernetes and AKS stages, each model segment is packaged as a separate service and communicates through synchronous gRPC forwarding. This makes the inference pipeline closer to a cloud-native microservice deployment, but it also introduces platform effects from container execution, service networking, CNI behavior, and managed-cluster infrastructure. Prior Kubernetes and cloud-performance studies show that these layers can affect measured latency and variability, which is why this thesis separates local process benchmarks, local Kubernetes benchmarks, and managed AKS benchmarks into distinct stages [6, 9, 17].

Chapter 3

Related Work

This chapter positions the thesis within prior work on split neural-network inference, cloud-native deployment overhead, and security hardening for microservice systems. The central distinction is that most prior partitioning work focuses on selecting or optimising split points, while this thesis follows one workload through a staged software-engineering evaluation: local split screening, Kubernetes deployment, managed AKS transfer validation, and security hardening.

3.1 Split Computing and DNN Partitioning

Early split-computing systems established that neural-network inference can be divided between execution environments, but that the split point determines whether the distributed design is useful. Neurosurgeon models the latency trade-off between mobile and cloud execution and shows that different layers have different computation and data transfer profiles [16]. JointDNN frames partitioning as an optimisation problem in which upload and download costs depend on the intermediate tensor at the chosen boundary [7]. DeeperThings extends this line of work to fully distributed CNN inference on constrained edge devices and reinforces the observation that early feature maps can be expensive to move because they remain spatially large [33].

More recent work broadens the partitioning objective beyond a single latency value. DNNSplit selects split points by jointly considering latency and cost in multi-tier settings [19]. Genetic-algorithm and delay-aware approaches likewise treat the split point as dependent on device capability, bandwidth, throughput, and parallelism constraints [20, 29]. Survey work on DNN partitioning and split computing summarizes this literature as a multi-objective problem involving activation size, compute allocation, deployment tier, and runtime conditions [24, 35]. This thesis adopts that view but keeps the optimisation scope deliberately narrow: it studies a fixed ResNet-18 workload and evaluates how a small number of architecturally meaningful split choices behave across deployment layers.

3.2 Granularity and Internal Model Structure

Several studies show that coarse layer depth is not enough to explain split performance. Fine-grained elastic partitioning demonstrates that more detailed placement decisions can improve distributed DNN inference when the placement is aware of both computation and communication conditions [31]. Hierarchical partitioning similarly combines coarse global structure with local refinement, showing that the useful split region can be narrowed before it is refined [34]. Pareto-front analysis of DNN partitioning further shows that execution time can vary across blocks inside the same model, so boundaries with similar activation sizes may still differ in compute distribution [21]. BottleFit and BottleNet++ approach the problem from the feature-compression side, showing that intermediate representations can be compressed or shaped to reduce communication cost, but that the position of the boundary remains central to the latency and accuracy trade-off [22, 32].

The experiments in this thesis use these findings to justify a coarse-to-fine design. RQ1.1 evaluates residual-stage boundaries in ResNet-18, and RQ1.2 adds a block-level boundary inside the carried-forward region. The goal is not to claim that the selected boundaries are globally optimal for all DNNs. Instead, the goal is to show how activation-transfer size and compute placement interact in a reproducible microservice setting.

3.3 Kubernetes and Cloud-Native Inference

Moving from local split inference to Kubernetes changes the systems context. A microservice boundary is not only a model partition; it is also an RPC path with serialization, protocol handling, socket operations, scheduling effects, and service networking. Recent RPC and microservice studies show that communication overhead can come from layered protocol processing and network-stack traversal, not only from the application payload itself [2, 39]. Service-mesh work reaches a similar conclusion for proxied microservices: sidecars and data-plane components add workload-dependent overhead through additional critical-path processing [38].

Kubernetes and cloud studies also show why local and managed-cluster results should be compared carefully. Managed Kubernetes performance varies across services and configurations [9]. Container and VM layers can differ from bare-metal execution [10]. CNI plugins and edge-oriented Kubernetes networking can affect communication cost [17]. Cloud environments also exhibit network-performance variability that can change application scalability and measurement stability [6]. For this reason, this thesis does not treat the local Kubernetes and AKS stages as numerical replicas. Instead, RQ1.5 asks whether the ordering and relative overhead trends from local Kubernetes transfer directionally to managed AKS.

3.4 Security Hardening

The security stages build on prior work in service identity, mTLS, service meshes, and confidential computing. Policy-driven Kubernetes security work motivates combining

encrypted communication with explicit authorization policy rather than treating TLS alone as the complete protection mechanism [4]. Service mesh studies show that sidecar-based mTLS can improve service-to-service trust boundaries, but also that it adds latency, CPU, memory, and operational complexity [5, 38]. Work on service-mesh control-plane trust further cautions that mesh CAs and control planes remain security-critical components [30]. RQ2.1 therefore treats mTLS and authorization as a bounded communication-hardening layer, not as a complete security solution.

Confidential-computing work motivates the second hardening layer. AMD SEV-SNP provides VM-level memory encryption and integrity protection against parts of the host platform [3, 18]. Comparative studies of virtualization-based confidential-computing platforms emphasize that these systems have specific attacker models, attestation mechanisms, and residual limitations [11, 27]. Performance studies show that confidential VM overhead is workload-dependent, with CPU-bound, memory-intensive, and I/O-heavy workloads affected differently [1, 36]. Machine-learning confidential computing surveys identify protected model execution and data privacy as important motivations, while also emphasizing deployment and performance trade-offs [28]. RQ2.2 follows this bounded framing by measuring VM-level confidential execution on AKS rather than claiming application-level enclave isolation.

3.5 Positioning of This Thesis

The related literature establishes the individual concerns that motivate this thesis: split points affect activation-transfer and compute costs, microservice and Kubernetes deployment add platform overhead, and security hardening strengthens protection while introducing additional cost. The contribution of this thesis is to connect those concerns in one staged empirical pipeline. Instead of evaluating partitioning, orchestration, and hardening as separate problems, the thesis tracks the same ResNet-18 microservice inference workload from local split selection through Kubernetes, AKS, mTLS/AuthZ, and confidential VM execution.

Chapter 4

Methods

This chapter describes the research design used to answer the research questions defined in Chapter 1. It explains which methods and procedures are used at each stage, why those methods are appropriate, and what the known limitations and pitfalls of the chosen approach are. The thesis addresses a problem that spans software architecture, containerised deployment, cloud infrastructure, and communication-level security. Studying these layers in a single undifferentiated experiment would confound the effects of partition boundary placement, orchestration overhead, and security mechanisms into one mixed result. The methodology therefore follows a deliberate *staged experimental pipeline* in which each stage changes one layer of the system at a time, preserves the model and benchmark contract, and compares against the nearest relevant baseline.

4.1 Overall Research Design

The thesis is organised around two research strands and seven empirical stages plus two synthesis stages. RQ1 studies how architectural split choice and deployment context affect microservice-based neural network inference. RQ2 studies the performance and operational cost of communication-level and runtime-level security hardening applied to the AKS deployment path selected from RQ1.

The stages unfold in the following sequence. RQ1.1 evaluates four coarse ResNet-18 stage boundaries under tightly controlled local CPU-only execution in order to identify which two-part decompositions are credible enough to carry forward before the thesis moves to orchestrated or cloud deployment. RQ1.2 tests one block-level split inside the carried-forward late-stage region to determine whether finer granularity reveals meaningful differences beyond the coarse level; it is a targeted refinement, not a new blind search. RQ1.3 is a hybrid analytical stage that explains the RQ1.1 and RQ1.2 latency patterns using two structural properties of the partition – activation-transfer size and compute distribution – drawing on the prior benchmark summaries and on a separately collected internal timing experiment on the monolithic model, without conducting a new split benchmark. RQ1.4 moves a predefined family of one-to-five service chains into a local single-node `kind` Kubernetes cluster to measure how end-to-end latency accumulates as synchronous service boundaries are added under in-cluster orchestra-

tion. RQ1.5 re-runs the same predefined chain family on Azure Kubernetes Service (AKS) to test whether the local Kubernetes ordering and overhead pattern transfer directionally to managed cloud execution. RQ2.1 adds managed-Istio mTLS, service accounts, and authorisation policy to the selected AKS deployment path, measuring the resulting latency, resource, and operational overhead using a paired execution design. RQ2.2 adds AMD SEV-SNP VM-level confidential execution to the already communication-secured chain from RQ2.1, measuring the incremental cost of selective and full two-service confidential-node placement. RQ2.3 synthesises the RQ2.1 and RQ2.2 results into a trade-off comparison across cost, operational complexity, and protection scope, without collecting any new benchmark data.

The common design principle across all stages is to change one layer of the system at a time. This principle follows benchmarking guidance that treats isolation of variables as the primary condition for interpretable systems performance results [13].

4.2 System Under Study

4.2.1 Model

The neural network used in all stages is ResNet-18 with pretrained ImageNet weights [12]. ResNet-18 is organised into four residual stages (`layer1` through `layer4`), each of which doubles the number of channels and halves the spatial resolution of the intermediate feature maps. This stage structure provides architecturally motivated split candidates: the four stage boundaries correspond to qualitatively distinct regions of the network with different activation sizes and different computational loads.

ResNet-18 is chosen for three reasons. First, its stage structure supports a principled coarse-to-fine screening methodology. Second, its size enables CPU-only inference at latencies realistic enough for measurement while avoiding the confounds of GPU scheduling variance in the early controlled stages. Third, as a standard reference architecture, its behaviour under partition is directly relatable to prior split-computing and DNN-partitioning work [16, 33, 35].

All experiments use FP32 precision with a fixed input shape of $1 \times 3 \times 224 \times 224$ and a batch size of one. Batch size one reflects the latency-oriented framing of the research questions: the thesis measures per-request end-to-end inference latency rather than throughput.

4.2.2 Microservice Framework

A proof-of-concept gRPC-based microservice framework was developed for this thesis. Each service runs as an independent process – or, in the Kubernetes stages, as an independent pod – that receives an input tensor over gRPC, executes its assigned segment of the ResNet-18 model, and transmits the resulting intermediate activation to the next service in the chain. The final service returns the classification output to a benchmark client.

gRPC is chosen as the inter-service protocol because it is a production-grade RPC framework with binary serialisation via Protocol Buffers, native streaming support, and

broad adoption in cloud-native deployments. Using gRPC ensures that the communication cost measured in this thesis reflects the kind of overhead expected in a realistic microservice inference pipeline rather than a hand-crafted minimal transport. The maximum gRPC message size is set to 16 MiB in all stages, which accommodates the largest intermediate activation tensor in the study (approximately 784 KiB for a split after `layer1` in FP32).

The framework supports variable chain depths: the same code path handles two-service chains used in RQ1.1, RQ1.2, and RQ2, and chains of up to five services used in RQ1.4 and RQ1.5. The independent variable in each stage is therefore the choice of split point or the number of chained services, not the communication mechanism.

4.3 Shared Benchmark Contract

To support direct comparison across stages, the thesis defines a shared benchmark contract that is preserved unless a stage explicitly states otherwise. The fixed subject under test is ResNet-18 with pretrained weights, executed on CPU in FP32 with input shape $1 \times 3 \times 224 \times 224$, communicating over gRPC on port 50051 with a maximum message size of 16 MiB.

The shared timing protocol consists of 50 warmup iterations followed by 200 measured iterations per condition execution, with a five-second cooldown between successive conditions, a fixed random seed of 42 for condition ordering within each round, and a warmup calibration check using a trailing window of 10 iterations and a coefficient-of-variation (CV) threshold of 0.02. Thesis-facing stages aggregate five rounds or five paired passes, yielding 1,000 measured iterations per condition. The warmup calibration procedure is not used to terminate warmup adaptively: its purpose is to provide evidence that the fixed 50-iteration warmup is sufficient for stable measurement in each execution environment.

The shared functional validation contract requires local segment validation before timing and live gRPC chain validation for any split or Kubernetes deployment, using an absolute tolerance of atol $= 1 \times 10^{-5}$ and five validation inputs. A condition is not benchmarked unless both checks pass. In practice, all thesis-facing frozen runs achieve a maximum absolute difference of 0.0 against the monolithic reference, confirming that no numerical drift is introduced by the partition or transport layer.

This shared contract is the invariant across the staged pipeline: what changes between stages is the infrastructure and security context, not the model, the measurement discipline, or the validation requirements.

4.4 CPU Stabilisation

Systematic CPU stabilisation is applied in all local and Kubernetes stages to reduce variance attributable to hardware noise rather than to the partition boundary. The following controls are requested in every thesis-facing local run. Thread counts for PyTorch intra-op and inter-op threads, and for OpenMP, MKL, and OpenBLAS, are all set to one, ensuring single-threaded model execution. CPU affinity is pinned to one physical core with simultaneous multithreading (SMT) excluded, preventing the benchmark process

from migrating between cores or sharing a physical core with a sibling hyperthread. Process scheduling priority is raised using `nice -5` where the runtime permits. The CPU frequency governor is set to `performance` and turbo boost is disabled to prevent frequency scaling from affecting measurements.

These controls are applied symmetrically across all conditions within a benchmark so that differences in measured latency can be attributed to the independent variable rather than to hardware variability.

In containerised and cloud stages, some controls are partially unavailable. In the local `kind` Kubernetes environment (RQ1.4), governor and turbo-disable writes to `/sys` are blocked by the read-only sysfs exposed inside the container; however, the detected governor is already `performance` and turbo is already disabled, so the intended state is achieved through the host configuration. In AKS (RQ1.5 and RQ2), the `nice -5` request is denied due to insufficient privileges and the benchmark runs at `nice = 0`; governor and turbo controls are unavailable through the managed node interface. These constraints are documented per stage and treated as realistic limitations of cloud-native deployment rather than as experimental flaws, because they affect all conditions within a stage symmetrically.

4.5 Measurement and Statistical Analysis

4.5.1 Primary Metrics

End-to-end latency is the primary measurement in all stages, defined as the wall-clock time for one full inference request from the moment the benchmark client dispatches the input to the moment it receives the final classification output.

Results are summarised using the arithmetic mean, median, and standard deviation of per-iteration latencies. The median provides a robust central tendency estimate in the presence of tail spikes. The mean is used as the primary comparison value because the thesis is concerned with expected per-request cost. The 95th-percentile (p95) latency is reported in the deployment and security stages to characterise tail behaviour under more realistic conditions.

Overhead is reported both in absolute milliseconds and as a percentage of the relevant baseline mean latency. In stages that compare conditions across environments (RQ1.5), within-environment normalised overhead is the primary comparison metric rather than raw absolute latency, because absolute values are environment-specific and do not transfer across different host CPUs, kernels, and virtualisation layers.

Several derived metrics are used in specific stages. The boundary-crossing interval captures the split-benchmark cost associated with crossing the service boundary, reflecting a composite of serialisation, transport, and the work executed on the remote side after the boundary. The activation-transfer burden is derived from the model architecture as the intermediate tensor size at a split boundary (or cumulative tensor size across all boundaries in a chain), not measured by a separate network profiler. The inferred non-compute/platform cost is a derived residual used in the Kubernetes, AKS, and mTLS analyses to separate platform and communication overhead from model computation analytically. In RQ2.1, resource and operational overhead metrics are also reported, including sidecar CPU and memory, total pod resource deltas, schedule-to-ready

time, and the count of added mesh and security objects. In RQ2.2, sequential throughput is derived from mean latency as requests per second to express the throughput impact of confidential-node placement.

4.5.2 Cross-Round Consistency

Cross-round consistency is assessed using the round CV, computed as the standard deviation of per-round condition means divided by the grand mean. A low round CV indicates stable execution across independent rounds and supports treating the reported condition means as representative rather than as artefacts of a single favourable or unfavourable run. Cross-round stability and parity validation are treated as the primary indicators of methodological validity.

4.5.3 Effect Size and Significance Tests

In stages with 1,000 per-condition measured iterations, Cohen's d is computed from pooled per-iteration data and the Mann-Whitney U test is reported for completeness. At $N = 1{,}000$ per condition, p-values are near-zero for any non-trivial difference and should not be over-interpreted as evidence of practical significance. Cohen's d provides the more informative measure: values below 0.2 are negligible, 0.2–0.5 are small, 0.5–0.8 are medium, and above 0.8 are large.

4.6 Carry-Forward and Selection Logic

The staged pipeline uses a carry-forward mechanism for the local split-selection stages (RQ1.1 and RQ1.2). After each of these stages, one or two candidates are selected to enter the next stage according to two predeclared rules.

The first rule concerns compute degeneracy. A split condition is classified as compute-degenerate if fewer than 10% of the total monolithic compute is allocated to the smaller of the two resulting services. Compute-degenerate conditions are excluded from main-candidate eligibility because they produce structurally unbalanced decompositions that do not constitute meaningful microservice partitions, even if their raw latency is low.

The second rule resolves near-ties among non-degenerate candidates. When the lowest and second-lowest raw latencies among eligible conditions fall within a 5% near-best window, the condition with the better structural balance – lower activation-transfer burden relative to its compute distribution – is selected as the main carry-forward candidate, and the other is retained as the reference candidate.

Carry-forward is not applicable in RQ1.4 and RQ1.5, which benchmark a predefined chain family rather than selecting among candidates. It is also not applicable in the RQ2 stages, which fix the deployment topology established by RQ1 and vary only the security configuration.

4.7 Stage-Specific Methods

4.7.1 RQ1.1 – Coarse Architectural Boundary Selection

RQ1.1 screens the four coarse stage boundaries of ResNet-18 against a monolithic baseline under the maximally controlled local setup described in Sections 4.4 and 4.3. The four split conditions place the boundary after `layer1`, `layer2`, `layer3`, and `layer4`, producing intermediate activation tensors of approximately 784 KiB, 392 KiB, 196 KiB, and 98 KiB in FP32 respectively. Both split services run as separate OS processes communicating via gRPC over `127.0.0.1:50051`.

The justification for screening at the coarse architectural level before conducting any finer search is that moving directly into an exhaustive layer-by-layer search would risk over-fitting the selection to incidental local-execution noise and would not align with the partitioning literature, which typically evaluates architecturally meaningful boundaries rather than arbitrary layers [19, 33, 35].

The output of this stage is one main carry-forward candidate and one reference candidate, selected using the degeneracy and near-best-window rules defined in Section 4.6.

4.7.2 RQ1.2 – Fine-Grained Refinement

RQ1.2 takes the carry-forward outcome of RQ1.1 as its starting point and introduces one block-level split point between the two coarse carry-forward anchors. The new condition, `split_after_layer3_block0`, bisects the `layer3` stage at its internal residual-block boundary.

A key structural property motivates this particular design choice. `split_after_layer3_block0` and `split_after_layer3` both produce intermediate activation tensors of approximately 196 KiB. The design therefore holds activation-transfer size constant while varying compute placement, creating a controlled comparison that can isolate the role of compute distribution independently of communication burden. This comparison is grounded in the DNN-partitioning literature, which treats compute distribution as a first-class criterion alongside activation-transfer cost [19, 31].

The experimental setup and measurement protocol are identical to RQ1.1, ensuring that the two stages remain directly comparable.

4.7.3 RQ1.3 – Explanatory Synthesis

RQ1.3 is not a split benchmark. It is a hybrid analytical stage that explains the patterns observed in RQ1.1 and RQ1.2 using two structural properties: activation-transfer size and compute distribution. The explanatory analysis draws on three evidence sources: the split benchmark summaries from RQ1.1, the split benchmark summaries from RQ1.2, and a separately conducted internal timing experiment on the monolithic ResNet-18 model.

The internal timing experiment instruments the monolithic forward pass at the stage, block, and selected operation level. It uses the same single-threaded CPU-stabilised environment as the split benchmarks, with 10 rounds, 50 warmup iterations per round,

and 200 measured iterations per round. Functional equivalence against the uninstrumented model is verified (`max_abs_diff` $= 0.0$), and instrumentation overhead is measured explicitly at approximately 0.645 ms (0.88% of total forward-pass time), confirming that profiling does not materially distort the timing distribution.

The justification for this supporting experiment is that split-benchmark boundary-crossing intervals reflect a composite of serialisation, transport, and remote-side computation. Disentangling those components – particularly distinguishing activation-transfer cost from compute-placement effects – requires a direct measurement of local per-block execution cost that the split benchmark alone cannot provide.

4.7.4 RQ1.4 – Multi-Microservice Kubernetes Chain

RQ1.4 moves from local process-based communication to real Kubernetes in-cluster services. The environment is a single-node `kind` cluster running on Ubuntu/Linux, with all service pods colocated on the same node. This colocation rule is applied deliberately so that inter-service gRPC traffic remains in-cluster rather than traversing an external network, meaning that any observed overhead reflects orchestration and pod-to-pod communication rather than wide-area latency.

The stage benchmarks a predefined family of five conditions from `monolithic_k8s_1svc` to `chain_5svc`, corresponding to one through five synchronously chained ResNet-18 service segments. The chain family is predefined rather than selected by carry-forward because the purpose of this stage is not to choose among candidates but to characterise the full cost curve as a function of service count. Service pods are allocated one CPU core and 512 MiB of memory with Guaranteed QoS to prevent throttling or migration during measurement windows.

The use of a single-node `kind` cluster is an acknowledged limitation: it eliminates inter-node network hops and does not capture the scheduling and networking variability of a multi-node or managed cluster. This limitation is addressed by the transfer-validation design of RQ1.5.

4.7.5 RQ1.5 – AKS Transfer Validation

RQ1.5 re-runs the same predefined chain family from RQ1.4 on a single-node AKS cluster (`Standard_D8s_v3`, region `swedencentral`, `kubenet` networking). The stage does not aim to replicate the local Kubernetes absolute latency values, which are expected to differ because the two environments use different host CPUs, kernels, virtualisation layers, container runtimes, and networking paths.

The transfer claim is therefore directional rather than numerical: if the same architectural chain ordering is preserved in AKS, that constitutes evidence that the ordering reflects the architecture of the model partition rather than incidental local-cluster noise [13]. Within-environment normalised overhead is used as the primary comparison metric for this reason.

All service pods and the benchmark client are colocated on the same AKS node and run with Guaranteed QoS, applying the same resource and placement discipline as RQ1.4.

4.7.6 RQ2.1 – Service Identity and Mutual TLS Hardening

RQ2.1 introduces a paired execution design. Each paired pass executes both the plain AKS condition and the mTLS/AuthZ condition in alternating order, with the order determined by a seeded plan in which three passes run mTLS first and two passes run plain first. This alternation reduces the risk that the observed delta is explained by slow performance drift over the course of the run rather than by the hardening mechanism itself.

The plain condition uses standard Kubernetes service-to-service communication without mesh injection. The mTLS condition adds one Istio sidecar proxy per inference service, service accounts, workload-level peer-authentication objects, and authorisation-policy objects. The benchmark client remains outside the mesh; the stage therefore measures hardening of the inter-service path inside the inference chain, not external ingress hardening.

The managed AKS Istio add-on (`asm-1-29`) is used as the mesh implementation. This choice reflects the thesis's focus on realistic cloud-native deployment: managed add-ons represent the configuration path most likely to be used by practitioners adopting mTLS in AKS and their behaviour reflects actual cloud-operator defaults rather than custom tuning.

Security validation is performed for each paired run using four methods: static manifest validation of peer-authentication and authorisation-policy objects; runtime validation of policy object counts; a full-chain positive probe confirming that valid upstream-to- downstream requests succeed; and a direct non-mesh denial probe confirming that unauthenticated access to protected downstream services is blocked.

The primary topology is `chain_2svc` (split after `layer2`), selected because it is the lowest-overhead non-monolithic chain in the preceding stages. A `chain_5svc` stress topology is included to determine how mTLS overhead grows when more service boundaries are protected.

A known limitation of sidecar-based mTLS is that it protects the inter-proxy communication path but does not prevent plaintext from existing inside the pod boundary after TLS termination. This limitation is acknowledged explicitly: RQ2.1 is claimed as a communication-level hardening layer, not as a complete inter-service secrecy mechanism [30].

4.7.7 RQ2.2 – Confidential VM Execution Hardening

RQ2.2 keeps the communication-security contract from RQ2.1 fixed and changes the execution environment of the inference services. The confidential condition places selected services on AMD SEV-SNP confidential node pools (`Standard_DC8as_v5`); the standard condition uses matched non-confidential AMD nodes (`Standard_D8as_v5`) in the same AKS cluster. Fresh paired standard baselines are collected for each RQ2.2 artifact set rather than reusing older RQ2.1 data, ensuring that incremental overhead can be attributed unambiguously to the confidential-node placement rather than to session-level drift.

Two configurations are evaluated. The selective TEE run places only the downstream service (`service2`) on confidential nodes, isolating the cost of protecting the service

that receives intermediate activations. The full TEE run places both services on confidential nodes, measuring the cost of extending the confidential-execution boundary to the entire two-service chain.

The protection boundary of AMD SEV-SNP must be stated precisely. SEV-SNP provides VM-level memory encryption and integrity protection against an untrusted host platform, but the guest operating system, application process, PyTorch runtime, model weights, and Istio sidecar all remain inside the same VM trust boundary. RQ2.2 therefore uses the term *VM-level confidential execution* throughout rather than claiming application-level enclave isolation.

4.7.8 RQ2.3 – Security-Hardening Trade-off Synthesis

RQ2.3 is a synthesis stage: no new benchmark data is collected. The stage combines the plain AKS baseline from RQ1.5, the mTLS/AuthZ results from RQ2.1, and the selective and full TEE results from RQ2.2 to compare the four hardening configurations across measured cost, operational complexity, and protection scope.

The synthesis is structured around the principle that the preferred hardening configuration is threat-model-dependent rather than universally optimal. Applying every available security mechanism to every service boundary is not always the best engineering choice, because each additional protection layer adds measurable latency, resource, and operational cost. RQ2.3 therefore frames its output as a structured trade-off comparison rather than a single global ranking, allowing a practitioner to match a security configuration to a stated threat model and performance budget.

4.8 Artifact Freezing and Reproducibility

Thesis-facing runs are exported and frozen as artifact sets before analysis is performed. This means that the reported results are derived from a fixed, immutable snapshot of the raw measurement data rather than from ad hoc terminal output or re-runnable live scripts. Each frozen artifact set records the full execution environment: operating system, Python and PyTorch version, Kubernetes version and cluster name, node type, namespace, container image digest, condition ordering, and parity validation outcomes.

The separation between the frozen artifact and the analysis code ensures that the reported numbers remain stable even if benchmark scripts are modified for subsequent exploratory runs. The repository contains both thesis-facing configs and older exploratory or smoke-test configs; the artifact freeze identifies which run is the thesis-facing primary result.

4.9 Limitations and Threats to Validity

Several methodological limitations should be acknowledged.

Single-node clusters. Both the RQ1.4 `kind` cluster and the RQ1.5 AKS cluster are single-node. Single-node deployments eliminate inter-node network hops and do not capture the scheduling and networking variability of a multi-node production cluster.

Results therefore reflect in-cluster pod-to-pod latency on a single physical or virtual machine and are not directly generalisable to distributed multi-node deployments.

CPU-only inference. All stages use CPU inference with a single thread. GPU-accelerated inference would substantially shift the compute distribution, reduce per-inference time, and change the relative weight of the gRPC boundary cost. The results are not directly generalisable to GPU-hosted microservice deployments.

Single model and workload. The thesis evaluates one model (ResNet-18) at batch size one with one input shape. Different architectures, larger activation maps, or higher-batch workloads may produce different cost curves and different optimal split regions.

Controlled conditions versus production variability. The local stages apply strict CPU stabilisation and achieve low cross-round variance. The Kubernetes and AKS stages are less controllable and exhibit higher variability. Neither environment captures the full noise of a shared multi-tenant cloud cluster, where CPU steal, network congestion, and noisy neighbours can substantially affect tail latency.

mTLS threat-model scope. Sidecar-based mTLS protects the inter-proxy communication path but does not prevent plaintext from existing inside the endpoint after TLS termination. The security claims for RQ2.1 are explicitly scoped to communication-level hardening.

SEV-SNP threat-model scope. AMD SEV-SNP protects the VM boundary against the host platform but not against a compromised guest OS or application process, and several side-channel and availability threats fall outside its core guarantee. RQ2.2 claims VM-level confidential execution, not complete application-level isolation.

4.10 Ethical Considerations

This thesis does not involve human participants, personal data, or biological material. The study is a software systems benchmark using a publicly available pretrained image classification model and standard synthetic inputs; no classification decisions derived from this model are used to affect any real-world outcomes, and no personal images or sensitive datasets are processed.

ResNet-18 with pretrained ImageNet weights inherits any biases or limitations of the ImageNet training process. However, since the thesis does not evaluate model accuracy and does not deploy the model in a decision-making context, these concerns are not directly relevant to the research questions addressed here.

Cloud compute resources are used responsibly: experiments are scoped to the minimum infrastructure required to answer each research question, and resources are deallocated after each benchmark stage. All measurement artifacts contain only timing data and system metadata; no sensitive data is stored on cloud infrastructure.

Chapter 5

Experiments & Results

5.1 RQ1.1 Coarse Architectural Boundary Selection

5.1.1 Research Question

This subsection addresses the following sub-research question:

> *Which coarse architectural stage boundaries provide the most suitable two-part microservice decompositions of ResNet-18 under controlled local execution, and how do their latency and boundary-crossing costs compare to monolithic inference?*

RQ1.1 is designed as a coarse-boundary screening stage rather than an exhaustive exploration of all possible split points. Its purpose is to identify architecturally meaningful two-part decompositions that can be compared against monolithic inference under controlled local execution, and to determine which coarse boundaries provide the strongest carry-forward basis for later refinement. In this stage, suitability is not treated as a question of raw latency alone. It is evaluated in relation to the joint trade-off between end-to-end latency, activation-transfer burden, boundary-crossing cost, and the structural balance of computation across the two resulting services.

5.1.2 Literature Context

Prior work on distributed and split neural network inference consistently shows that partitioning a model across execution boundaries introduces a fundamental trade-off between local computation and communication overhead. DNNSplit demonstrates that split points should be selected systematically with respect to latency and cost, rather than by arbitrary layer choice [19]. More broadly, DeeperThings shows that split-boundary placement strongly affects performance because early feature maps are often large and expensive to transmit, while later layers tend to reduce activation size but can shift too much computation to one side of the partition [33].

Survey literature on deep neural network partitioning across cloud, edge, and end devices likewise emphasizes that effective partitioning requires balancing activation-transfer cost, compute distribution, and deployment constraints rather than focusing solely on network depth [35]. Optimization-based approaches such as genetic algorithms further suggest that the most suitable boundary is context-dependent and should be judged with respect to latency, bandwidth, and device capability [29]. These studies support the view that partitioning is a multi-objective problem rather than a purely architectural one.

Recent work has extended this perspective by considering finer-grained or adaptive partitioning strategies. Fine-grained elastic partitioning for distributed DNN inference over 5G networks shows that performance can improve when boundaries are selected with awareness of communication conditions and execution constraints [31]. Delay-aware scheduling studies likewise show that inference latency and throughput are sensitive to where and how the model is split, especially in heterogeneous edge environments [20]. More recent hierarchical partitioning studies similarly indicate that different network regions may be preferable depending on resource availability and the desired latency–communication trade-off [34].

Although much of this literature is framed in device–edge–cloud settings rather than microservice-based inference, the underlying principles remain relevant. A microservice boundary introduces the same core costs of serialization, transmission, and deserialization, even when the communicating services run on the same host. For that reason, the stage-level decomposition studied here is grounded in the partitioning literature but adapted to a controlled local microservice setting.

5.1.3 Experimental Setup

The RQ1.1 benchmark used the fully controlled local CPU-only configuration in order to minimize variance and isolate the effect of the partition boundary itself. The evaluated model was ResNet-18 with pretrained weights, executed on CPU in FP32 precision with an input shape of $1 \times 3 \times 224 \times 224$. The benchmark compared one monolithic baseline against four two-part split configurations, each corresponding to a coarse architectural stage boundary in ResNet-18.

Table 5.1: RQ1.1 benchmark configuration

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Conditions	Monolithic + split after layer1, layer2, layer3, layer4
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC over localhost (`127.0.0.1:50051`)
Maximum message size	16 MB
Parity tolerance	1.0×10^{-5}
Parity inputs	5
Warmup window	10 iterations
Warmup CV threshold	0.02

The benchmark was configured to use a single-threaded PyTorch execution model pinned to one physical CPU core with SMT avoided. Thread counts for PyTorch, OpenMP, MKL, and OpenBLAS were all fixed to one. In addition, process priority was increased, the CPU governor was set to `performance`, and turbo boost was disabled in order to reduce frequency-related variance. These controls were applied symmetrically across all conditions so that the comparison remained fair and reproducible.

5.1.4 Benchmark Design

The experiment followed a repeated-measures design with five conditions: a monolithic baseline and four split configurations. The split points were selected after the major architectural stages of ResNet-18, namely after `layer1`, `layer2`, `layer3`, and `layer4`. These boundaries are meaningful because they preserve the stage structure of the network while allowing the evaluation of progressively later partition points.

The benchmark protocol consisted of 5 rounds. For each round and condition, 50 warmup iterations were executed before 200 measured iterations were recorded. A 5-second cooldown separated successive conditions. The ordering of conditions was randomized using a fixed seed to reduce ordering bias. Functional equivalence between monolithic and split executions was enforced as a mandatory precondition through both local and gRPC parity checks using five inputs and an absolute tolerance of 1.0×10^{-5}.

A warmup calibration procedure was used to monitor whether latency had stabilized. This procedure used a trailing window of 10 iterations and a coefficient-of-variation threshold of 0.02. The goal was not to terminate warmup adaptively, but to provide evidence that the fixed warmup count was sufficient for stable measurement.

5.1.5 Partition Conditions

The four split conditions corresponded to the following coarse stage boundaries in ResNet-18:

- **split_after_layer1:** The first service executes the stem and `layer1`, and the second service executes `layer2` through the classifier head. This split transfers a large intermediate tensor, approximately $64 \times 56 \times 56$, corresponding to roughly 784 KB in FP32.
- **split_after_layer2:** The first service executes the stem and `layer1--layer2`, and the second service executes `layer3` through the classifier head. The intermediate tensor is approximately $128 \times 28 \times 28$, or 392 KB.
- **split_after_layer3:** The first service executes the stem and `layer1--layer3`, and the second service executes `layer4` plus pooling and classification. The intermediate tensor is approximately $256 \times 14 \times 14$, or 196 KB.
- **split_after_layer4:** The first service executes the stem and `layer1--layer4`, and the second service executes only average pooling and the final linear layer. The intermediate tensor is approximately $512 \times 7 \times 7$, or 98 KB.

These stage boundaries provide a coarse sweep across the network from earlier to later partition points. This design is consistent with partitioning research that typically evaluates a limited set of architecturally meaningful split points rather than all layers exhaustively [19, 33, 35].

5.1.6 Results

The measured results show that monolithic execution achieved the lowest mean end-to-end latency at 76.12 ms. The four split conditions yielded mean latencies of 79.65 ms for

split_after_layer1, 78.63 ms for split_after_layer2, 78.50 ms for split_after_layer3, and 77.26 ms for split_after_layer4. This corresponds to relative overheads of approximately 4.6%, 3.3%, 3.1%, and 1.5%, respectively, compared to monolithic execution.

Table 5.2: RQ1.1 summary of mean latency and overhead

Condition	Mean (ms)	Median (ms)	Std. Dev. (ms)	Overhead vs. Monolithic
Monolithic	76.12	74.04	3.89	–
Split after layer1	79.65	78.77	2.28	4.6%
Split after layer2	78.63	77.65	2.73	3.3%
Split after layer3	78.50	77.56	2.74	3.1%
Split after layer4	77.26	76.09	2.65	1.5%

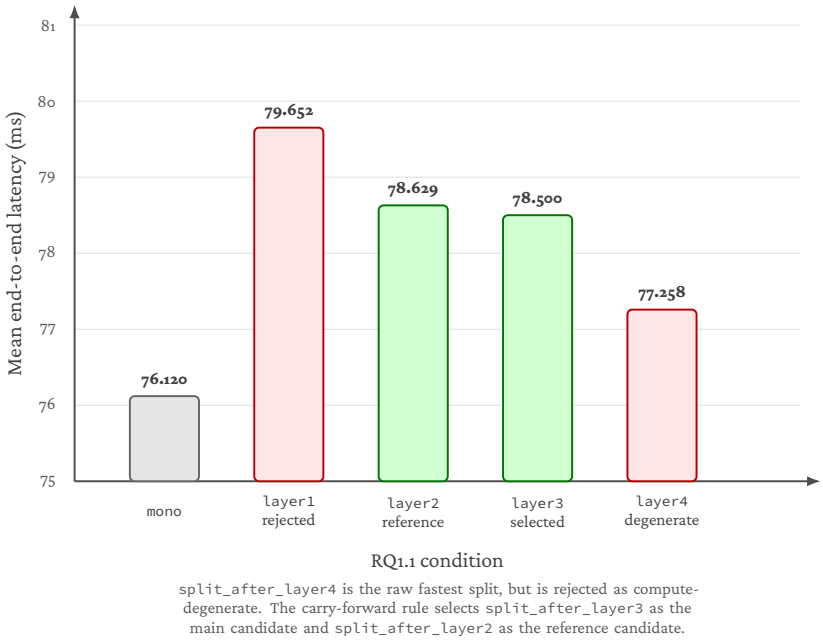


Figure 5.1: RQ1.1 coarse split latency results under controlled local execution. Although split_after_layer4 has the lowest mean latency among split configurations, it is rejected by the compute-degeneracy rule. The selected carry-forward candidates are split_after_layer3 and split_after_layer2.

Figure 5.1 visualises the latency pattern reported in table 5.2. The raw fastest split is the latest boundary, but the carry-forward decision depends on both latency and structural compute balance rather than latency alone.

All split configurations therefore introduced positive latency overhead relative to monolithic inference. However, the magnitude and character of this overhead varied substantially with split location. The earliest split, after layer1, performed worst be-

cause it crossed the largest intermediate activation and exhibited the highest boundary-crossing cost. The latest split, after `layer4`, minimized activation-transfer size and achieved the lowest raw split latency. However, it left only a minimal residual computation segment on the second service (under 1 ms), making it structurally compute-degenerate despite its low transfer burden. The mid-to-late splits, after `layer2` and `layer3`, formed the strongest non-degenerate candidates in the coarse sweep.

The boundary-crossing measurements further support this interpretation. The split after `layer1` incurred the largest boundary-crossing interval at 53.03 ms, followed by `layer2` at 39.96 ms and `layer3` at 25.73 ms, while `layer4` was much smaller at 2.13 ms. This descending pattern is consistent with the reduction in activation-transfer burden from 784 KB at `layer1` to 98 KB at `layer4`. At the same time, the late `layer4` split shows that minimizing transfer burden alone does not determine overall suitability as a microservice decomposition, because the downstream service becomes too small to represent a meaningful balanced distribution.

Cross-round consistency was strong across all configurations, indicating stable execution behavior. The monolithic condition had a round coefficient of variation (CV) of 0.013, while `split_after_layer2`, `split_after_layer3`, and `split_after_layer4` had round CVs of 0.007, 0.008, and 0.008, respectively. The split after `layer1` also remained stable with a round CV of 0.009.

5.1.7 Interpretation

The observed pattern aligns well with prior literature on split computing and distributed CNN inference. First, the poor performance of the earliest split is consistent with the widely reported effect that early feature maps are expensive to transmit because they remain large in spatial dimensions [33, 35]. Second, the late split after `layer4` shows that minimizing activation size alone is not sufficient; a split must also preserve a meaningful compute balance across services [21, 23]. Third, the favorable position of `split_after_layer2` and `split_after_layer3` is consistent with prior work that often identifies mid-network partitions as plausible trade-offs between communication and computation [19, 21, 33].

The benchmark also indicates that raw latency ranking alone is not sufficient for the thesis-facing carry-forward decision. Under the predeclared selection rule, the lowest raw split latency was observed at `split_after_layer4`. However, this candidate was excluded from main-candidate eligibility because it was compute-degenerate. Among the remaining non-degenerate candidates, `split_after_layer3` and `split_after_layer2` presented extremely similar end-to-end latencies. Through the defined carry-forward resolution, `split_after_layer3` is selected as the main carry-forward candidate and `split_after_layer2` is retained as the reference candidate.

The results therefore support the thesis hypothesis that coarse architectural stage boundaries are not equally suitable for microservice-based inference. Rather, the most suitable boundaries are those that reduce activation-transfer burden without collapsing the second partition into a near-empty remainder. In this experiment, the strongest balanced carry-forward region lies in the mid-to-late coarse portion of the network around `layer2` and `layer3`, while the latest split after `layer4` remains informative as the raw-fastest but structurally degenerate boundary.

5.1.8 Answer to RQ1.1

RQ1.1 shows that coarse architectural stage boundaries are not equally suitable for two-part microservice decomposition of ResNet-18 under controlled local execution. Early boundaries incur high activation-transfer and boundary-crossing costs, while the latest boundary after `layer4` minimizes transfer size but becomes compute-degenerate. The most suitable balanced carry-forward candidates therefore lie in the mid-to-late coarse region around `layer2` and `layer3`, which provide the most credible trade-off between end-to-end latency, communication-related cost, and compute distribution. According to the screening logic, `split_after_layer3` is selected as the main coarse carry-forward candidate, with `split_after_layer2` retained as the reference candidate [19, 21, 33, 35].

5.2 RQ1.2 Fine-Grained Refinement

5.2.1 Research Question

This subsection addresses the following sub-research question:

> *How does finer-grained refinement within the late-stage coarse region carried forward from RQ1.1 affect latency and communication-related overhead for two-part microservice inference?*

RQ1.2 is designed as a targeted refinement stage, not as a new blind search across the network. It takes the carry-forward outcome of RQ1.1 as its starting point. In RQ1.1, the coarse architectural boundary screening identified the mid-to-late region around `layer2` and `layer3` as the strongest balanced carry-forward zone, with `split_after_layer3` selected as the main candidate and `split_after_layer2` retained as the reference candidate. RQ1.2 therefore operates within that carried-forward comparison space and introduces one meaningful finer-grained split point between the two coarse anchors. The purpose is to determine whether a block-level boundary within this region reveals latency or communication-related differences that were not visible at the coarse stage level.

5.2.2 Literature Context

The broader DNN partitioning literature consistently identifies partition granularity as a factor that influences the latency–communication trade-off. Neurosurgeon was among the first systems to demonstrate that layer-level partitioning between a mobile device and the cloud systematically outperforms coarse offloading strategies, because different layers exhibit different computational and communication profiles [16]. This finding implies that two split points that appear similar at a coarse level may differ once the internal structure of the intervening layers is considered.

DNNSplit extends this perspective by showing that split points should be selected with respect to both latency and cost rather than by arbitrary layer choice, and that the optimal boundary depends on the characteristics of the specific network and deployment context [19]. DeeperThings similarly shows that early feature maps are large and expensive to transmit, while later boundaries reduce activation size but may shift the compute balance too far toward one side of the partition [33]. The DNN partition survey by Xu et al. reinforces that effective partitioning requires jointly considering activation-transfer cost, compute distribution, and the granularity at which boundaries are placed [35].

More directly relevant to RQ1.2 is the work by Ren et al. on fine-grained elastic partitioning for distributed DNN inference in 5G networks [31]. That study explicitly contrasts coarse single-point partitioning with finer-grained multi-point partitioning and shows that the latter can meaningfully improve inference latency when boundaries are placed with awareness of per-layer computation and communication characteristics. Although their setting involves edge–cloud collaboration rather than local microservice inference, the underlying principle carries over: a single coarse partition may miss internal structure that a refined boundary can expose.

Hierarchical partitioning strategies provide further support for this reasoning. HiDP proposes a two-tier partitioning approach that first creates coarse global blocks and then refines them locally, and shows that the hierarchical strategy achieves lower latency than approaches confined to global-level partitioning alone [34]. This two-tier logic parallels the relationship between RQ1.1 and RQ1.2 in this thesis, where coarse screening is followed by block-level refinement within the selected region.

Recent work on block-wise profiling of DNN models by Masud et al. further demonstrates that execution time varies substantially across blocks within the same model, suggesting that not all layers with similar depth are computationally equivalent [21]. This observation is directly relevant to the central hypothesis of RQ1.2: that two split points producing the same intermediate activation size may still differ in end-to-end latency because the distribution of computation around the boundary differs. BottleFit similarly shows that the position of a bottleneck within a network affects not only the compression rate but also the resulting accuracy and processing cost, confirming that boundary placement involves trade-offs beyond activation size alone [22].

Taken together, these studies establish that partition granularity is not merely a design convenience but a factor with measurable performance consequences. RQ1.2 applies this principle within the specific carried-forward region from RQ1.1 in order to determine whether block-level refinement reveals meaningful differences beyond the coarse stage-boundary comparison.

5.2.3 Experimental Setup

The RQ1.2 benchmark reused the same fully controlled local CPU-only configuration as RQ1.1 in order to ensure that the two stages remain directly comparable. The evaluated model was ResNet-18 with pretrained weights, executed on CPU in FP32 precision with an input shape of $1 \times 3 \times 224 \times 224$. Communication between the two split services used gRPC over localhost (`127.0.0.1:50051`) with a maximum message size of 16 MB.

All CPU stabilisation controls from RQ1.1 were applied identically: single-threaded PyTorch execution pinned to one physical core with SMT avoided, thread counts for PyTorch, OpenMP, MKL, and OpenBLAS all fixed to one, process priority elevated, the CPU governor set to `performance`, and turbo boost disabled. These controls were applied symmetrically across all conditions so that any observed differences can be attributed to boundary placement rather than to environmental variation.

5.2.4 Benchmark Design

The experiment followed the same repeated-measures protocol as RQ1.1: 5 rounds of 200 measured iterations per condition, preceded by 50 warmup iterations and separated by 5-second cooldowns. Condition ordering was randomised within each round using a fixed seed. Functional equivalence between monolithic and split executions was enforced through both local and gRPC parity checks using five inputs and an absolute tolerance of 1.0×10^{-5}. A warmup calibration procedure using a trailing window of 10 iterations and a coefficient-of-variation threshold of 0.02 was applied to verify that latency had stabilised before measurement.

Table 5.3: RQ1.2 benchmark configuration

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Conditions	Monolithic + split after layer2, layer3.0, layer3
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC over localhost (`127.0.0.1:50051`)
Maximum message size	16 MB
Parity tolerance	1.0×10^{-5}
Parity inputs	5
Warmup window	10 iterations
Warmup CV threshold	0.02

The key difference from RQ1.1 is the composition of the condition set. Rather than sweeping across all coarse stage boundaries, RQ1.2 evaluates a targeted set of three refined split conditions anchored by the RQ1.1 carry-forward candidates, with one new block-level split point inserted between them.

5.2.5 Refined Partition Conditions

The RQ1.2 condition set consists of one monolithic baseline and three split configurations:

- **split_after_layer2:** The first service executes the stem and `layer1`–`layer2`, and the second service executes `layer3` through the classifier head. This is the RQ1.1 reference carry-forward candidate. The intermediate activation tensor is approximately $128 \times 28 \times 28$, or 392 KB in FP32.
- **split_after_layer3_blocko:** The first service executes the stem and `layer1`–`layer3.0` (i.e., the first residual block of `layer3`), and the second service executes from `layer3.1` onward through the classifier head. This is the single new finer-grained split point introduced in RQ1.2. It bisects the coarse `layer3` stage at its internal block boundary. The intermediate activation tensor is approximately $256 \times 14 \times 14$, or 196 KB in FP32 — the same size as the tensor produced at the end of the full `layer3` stage.
- **split_after_layer3:** The first service executes the stem and `layer1`–`layer3`, and the second service executes `layer4` plus pooling and classification. This is the RQ1.1 main carry-forward candidate. The intermediate activation tensor is approximately $256 \times 14 \times 14$, or 196 KB in FP32.

Table 5.4: RQ1.2 summary of mean latency, overhead, and activation-transfer burden

Condition	Mean (ms)	Median (ms)	Std. Dev. (ms)	Overhead	Activation (KB)
Monolithic	76.291	73.778	3.578	—	—
Split after layer2	78.689	77.615	2.669	3.1%	392
Split after layer3_block0	78.691	77.999	2.715	3.1%	196
Split after layer3	78.853	77.836	2.677	3.4%	196

The monolithic baseline is retained for reference so that all overhead values remain anchored to the same absolute comparison. Together, these four conditions form a targeted refinement set rather than an exhaustive within-stage search: `split_after_layer2` and `split_after_layer3` anchor the carried-forward coarse space, while `split_after_layer3_block0` probes the single meaningful block-level internal boundary between them.

An important structural observation motivates this design. The coarse boundary after `layer3` and the block-level boundary after `layer3.0` produce intermediate activation tensors of the same size (both approximately 196 KB). This creates a controlled comparison in which the activation-transfer burden is held constant while the distribution of computation across the two services differs. If the two conditions yield different end-to-end latencies, the difference can be attributed to compute placement rather than to communication cost.

5.2.6 Results

The measured results are summarised in table 5.4. Monolithic execution achieved the lowest mean end-to-end latency at 76.291 ms. Among the three refined split conditions, `split_after_layer2` achieved the lowest mean split latency at 78.689 ms, followed by `split_after_layer3_block0` at 78.691 ms and `split_after_layer3` at 78.853 ms. This corresponds to overhead relative to monolithic execution of approximately 3.1%, 3.1%, and 3.4%, respectively.

The boundary-crossing interval measurements reveal a notable difference between the two conditions that share the same activation size. `split_after_layer3_block0` exhibited a boundary-crossing interval of 33.135 ms, while `split_after_layer3` exhibited a boundary-crossing interval of 25.718 ms. Despite producing activation tensors of the same size (approximately 196 KB), the two conditions differ by more than 7.4 ms in boundary-crossing cost. The earlier split (`layer2`) had the largest boundary-crossing interval at 39.937 ms, consistent with its larger activation tensor of 392 KB.

Cross-round consistency was strong across all conditions. The monolithic condition had a round coefficient of variation (CV) of 0.001, while the split conditions had round CVs of 0.003 (`split_after_layer2`), 0.004 (`split_after_layer3`), and 0.015 (`split_after_layer3_block0`). The slightly higher variability of `split_after_layer3_block0` is attributable to a favourable first round; from round 2 onward its per-round means were consistent with the other split conditions.

5.2.7 Interpretation

The results support three observations.

The refined region remains tightly clustered. All three refined split conditions fall within a very narrow latency band, from 78.689 ms to 78.853 ms. This indicates that the block-level refinement does not reveal a dramatically superior new split. Instead, it confirms that the carried-forward `layer2`–`layer3` region identified in RQ1.1 already captures the main structure of the latency–communication trade-off in this late-stage part of the network.

Equal activation size does not guarantee equal performance. The most instructive comparison in this refinement stage is between `split_after_layer3_block0` and `split_after_layer3`. Both conditions transfer an intermediate tensor of approximately 196 KB, yet they differ in boundary-crossing interval by more than 7.4 ms and in mean end-to-end latency by a smaller but still measurable amount. This demonstrates that activation-transfer size is not the sole determinant of boundary cost. The distribution of computation on either side of the boundary also matters: shifting the split from the end of `layer3` to the internal boundary after `layer3.0` changes the compute placement and therefore changes boundary-crossing behavior even when the serialized tensor is the same size. This finding is consistent with the broader partitioning literature, which treats compute distribution as a first-class consideration alongside communication cost [19, 31, 33–35].

This run does not overturn the coarse screening. Under the current refined benchmark outcome, `split_after_layer2` is the raw-fastest split and is selected as the main candidate by the implemented carry-forward rule, with `split_after_layer3_block0` retained as the reference candidate. However, the three refined conditions remain so close that the central thesis-facing conclusion is not that RQ1.2 discovered a dramatically better split than RQ1.1. Rather, the refinement confirms that the late carried-forward region remains the correct focus, while revealing that small block-level shifts can still change behavior measurably even when activation-transfer size is held constant.

5.2.8 Answer to RQ1.2

RQ1.2 shows that finer-grained refinement within the carried-forward coarse region from RQ1.1 does not overturn the coarse-level findings, but it does reveal a meaningful secondary insight. In the latest refined comparison, `split_after_layer2` is the raw-fastest split and is selected as the main candidate under the implemented selection rule, while `split_after_layer3_block0` is retained as the reference candidate. The refined split `split_after_layer3` is slightly slower in this run. At the same time, the comparison between `split_after_layer3_block0` and `split_after_layer3` — which share the same activation-transfer size of approximately 196 KB yet differ in boundary-crossing interval by more than 7.4 ms — demonstrates that equal activation-transfer size does not guarantee equal performance. Compute placement within the refined region still matters, even when the serialized tensor is held constant. The coarse

RQ1.1 screening had already identified the correct late-stage comparison space; the block-level refinement confirms that structure and adds the observation that per-block compute distribution is a secondary but observable factor in boundary-crossing cost [16, 19, 22, 31, 33–35].

5.3 RQ1.3 Explanatory Synthesis: Activation-Transfer Size and Compute Distribution

5.3.1 Research Question

This subsection addresses the following sub-research question:

> *How do activation-transfer size and compute distribution explain the latency and boundary-crossing behavior observed for the decomposition choices evaluated in RQ1.1 and RQ1.2?*

RQ1.3 is not an additional split benchmark. It is an explanatory synthesis stage that interprets the local findings already reported in RQ1.1 and RQ1.2 using two structural properties of the partition: the size of the intermediate activation tensor transferred at the boundary, and the distribution of computation across the two resulting services. To support this interpretation, RQ1.3 draws on the split benchmark measurements from RQ1.1 and RQ1.2 as primary evidence and uses an independently conducted internal timing experiment on the monolithic ResNet-18 model as supporting evidence for the compute-distribution explanation.

5.3.2 Literature Context

The literature on distributed and split DNN inference provides a well-established basis for the two explanatory dimensions used in this section.

The role of intermediate activation size as a determinant of communication cost has been recognised since the earliest work on DNN partitioning. Neurosurgeon demonstrated that the data volume transferred at the split boundary varies substantially across layers and that this variation directly affects end-to-end latency [16]. JointDNN extends this insight by modelling the partition decision at layer granularity as a graph optimisation problem in which upload and download costs at each candidate boundary are determined by the size of the intermediate tensor, confirming that communication cost is proportional to the activation footprint of the chosen boundary [7]. BottleNet++ provides further evidence by showing that earlier split points produce substantially larger intermediate representations, requiring dedicated feature-compression architectures to make those boundaries viable; without compression, early splits incur communication costs that dominate end-to-end latency [32]. DeeperThings similarly observes that early feature maps are spatially large and expensive to transmit, while later boundaries reduce the activation footprint at the cost of shifting the compute balance [33]. The DNN partitioning survey by Xu et al. reinforces this picture by noting that activation-transfer overhead is one of the primary cost components that any partition strategy must account for [35].

The complementary role of compute distribution has received growing attention. DNNSplit argues that optimal split-point selection requires jointly minimising latency and cost rather than focusing on transfer size in isolation, because the computational load on each side of the boundary affects wall-clock performance independently of communication overhead [19]. DADS (Dynamic Adaptive DNN Surgery) demonstrates that

the intermediate data size at certain DNN layers can be significantly smaller than the raw input, but that exploiting this property requires awareness of the per-layer compute characteristics on both sides of the partition; the optimal split point shifts dynamically depending on the computational and network context [14]. The Pareto-front analysis by Masud et al. provides direct evidence that execution time varies substantially across blocks within the same model, even when those blocks process tensors of similar size [21]. BottleFit further shows that the position of a split within the network changes not only the size of the intermediate representation but also the processing cost surrounding it [22].

The idea that these two dimensions should be considered jointly is a recurring theme. Fine-grained elastic partitioning demonstrates that performance improves when boundaries are placed with awareness of both per-layer computation and communication characteristics [31]. More broadly, Matsubara and Levorato observe that split computing involves an inherent tension between the computational asymmetry of the two partitions and the bandwidth cost of the intermediate representation [23]. The edge intelligence survey by Zhou et al. situates these trade-offs within a broader architectural taxonomy of device–edge–cloud collaborative inference, noting that the choice of split point fundamentally determines both the communication overhead and the compute allocation across tiers [37]. These studies collectively support the explanatory framework adopted in RQ1.3: activation-transfer size and compute distribution are not alternative explanations but complementary dimensions that jointly determine the latency and boundary-crossing behavior of a partitioned model.

Finally, the internal timing analysis used as supporting evidence in this section is grounded in the architectural design of ResNet-18 itself. He et al. describe a stage-based residual architecture in which each stage doubles the channel count and halves the spatial resolution, producing an uneven distribution of FLOPs and memory across stages [12]. This architectural property means that per-stage and per-block compute costs are structurally non-uniform, which is directly relevant to explaining why different boundaries yield different performance even when they fall in the same region of the network.

5.3.3 Analytical Setup

RQ1.3 draws on three bodies of evidence:

1. **Split benchmark measurements from RQ1.1.** These include the mean end-to-end latencies, boundary-crossing intervals, and activation-transfer sizes for the five coarse conditions (monolithic plus four stage-level splits). The relevant data are summarised in table 5.2 and the overhead analysis in section 5.1.6.
2. **Split benchmark measurements from RQ1.2.** These include the mean end-to-end latencies, boundary-crossing intervals, and activation-transfer sizes for the four refined conditions (monolithic plus three refined splits within the carried-forward region). The relevant data are summarised in table 5.4 and the overhead analysis in section 5.2.6.
3. **Internal timing experiment on monolithic ResNet-18.** A separate instrumented forward-pass profiling experiment was conducted on the same monolithic model under a closely matched single-threaded CPU-stabilised environment. This ex-

Table 5.5: RQ1.3 supporting internal timing configuration

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Profiling mode	Monolithic internal timing (supporting evidence)
Timing scope	Stage-, block-, and selected operation-level timing
Rounds	10
Warmup iterations	50
Measured iterations	200
Validation inputs	5
Validation tolerance	1.0×10^{-6}
CPU stabilisation	Single-threaded, single-core pinned execution
Instrumentation overhead	0.645 ms (0.88%)

periment measured per-stage, per-block, and selected internal operation timings for a single-threaded CPU forward pass, averaged over 200 measured iterations preceded by 50 warmup iterations, using 10 rounds and the same randomisation seed. Functional equivalence was verified (maximum absolute difference of 0.0 against the uninstrumented model), and instrumentation overhead was measured at 0.645 ms (0.88% of total forward-pass time), confirming that the profiling did not meaningfully distort the timing distribution. This experiment is used as supporting evidence to characterise the internal compute distribution of ResNet-18; it is not a split benchmark and does not introduce new split conditions.

No new split benchmark was conducted for RQ1.3. The explanatory analysis is based entirely on measurements already reported in RQ1.1 and RQ1.2, supplemented by the internal timing experiment described above. To keep this supporting evidence auditable without turning RQ1.3 into a separate benchmark stream, tables 5.5 to 5.7 and fig. 5.2 provide a compact summary of the internal timing configuration and the specific internal timing results used in the explanatory analysis.

5.3.4 Explanatory Findings

The findings are organised around four explanatory observations, each building on the previous one.

A. Activation-transfer burden explains much of the coarse latency pattern. In RQ1.1, the boundary-crossing interval decreased monotonically from the earliest split to the latest: 53.03 ms after `layer1` (784 KB activation), 39.96 ms after `layer2` (392 KB), 25.73 ms after `layer3` (196 KB), and 2.13 ms after `layer4` (98 KB). This pattern closely tracks the reduction in activation-transfer size as the split point moves deeper into the network. Larger intermediate tensors require more serialisation, transmission, and deseri-

Table 5.6: RQ1.3 supporting internal timing summary used in the explanatory analysis

Unit / Metric	Mean (ms)	% of model
Model total	73.753	100.00%
Stem	12.028	16.31%
Layer1	13.071	17.72%
Layer2	11.968	16.23%
Layer3	13.689	18.56%
Layer4	22.639	30.70%
Layer3.1 block	7.340	9.95%
Classification head	0.324	0.44%
Convolution ops	—	81.8%
BatchNorm ops	—	3.3%

Table 5.7: RQ1.3 heaviest internal compute units from the supporting timing analysis

Unit	Mean (ms)	% of model
`stem.maxpool`	7.189	9.75%
`layer4.1.conv1`	6.039	8.19%
`layer4.0.conv2`	6.029	8.17%
`layer4.1.conv2`	5.972	8.10%
`layer3.0.conv2`	3.605	4.89%
`stem.conv1`	3.598	4.88%
`layer3.1.conv1`	3.576	4.85%
`layer4.0.conv1`	3.485	4.73%

alisation time over the gRPC boundary, and this cost is directly reflected in the boundary-crossing measurements.

This observation is consistent with the broader partitioning literature, which identifies activation size as a primary driver of communication overhead in distributed inference [7, 16, 32, 33, 35]. At the coarse level, the monotonic relationship between activation size and boundary-crossing interval confirms that transfer burden is the dominant explanatory factor for the boundary-crossing cost gradient observed across the four RQ1.1 split points.

B. Activation size alone does not determine suitability. Although `split_after_layer4` achieved the lowest boundary-crossing interval and the lowest raw split latency in RQ1.1 (77.26 ms), it was classified as compute-degenerate because the second service executed only average pooling and the final linear layer — a residual computation segment of well under 1 ms. This means that minimising activation-transfer size to its smallest value does not, by itself, produce a suitable microservice decomposition. A boundary that pushes nearly all computation to one side leaves the second service structurally empty, which defeats the purpose of decomposition.

The internal timing experiment provides quantitative support for this observation.

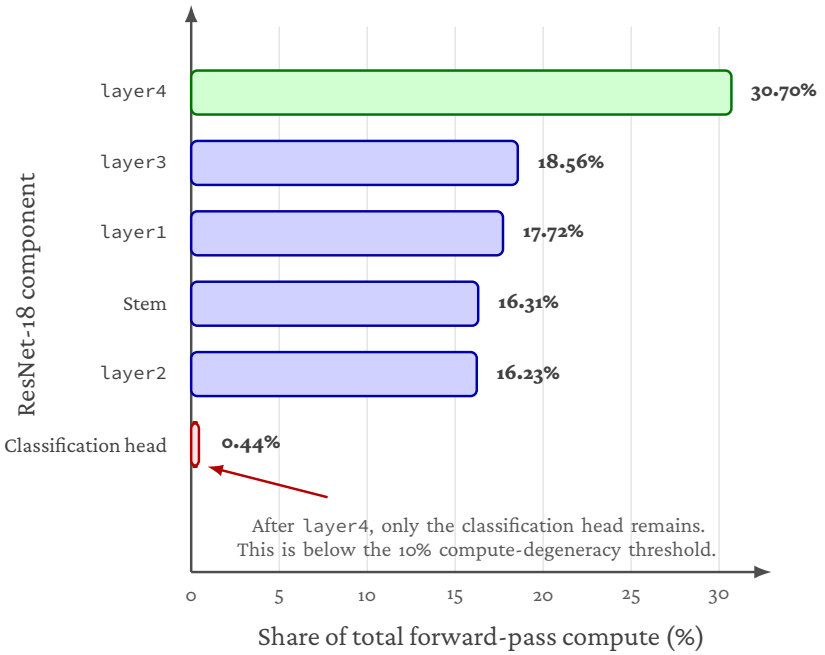


Figure 5.2: Stage-level compute distribution from the RQ1.3 internal timing analysis. The final classification head contributes less than 1% of total forward-pass compute, while `layer4` accounts for the largest stage-level share. This explains why `split_after_layer4` is raw-fast but compute-degenerate: almost all meaningful model computation remains before the split.

As shown in fig. 5.2, the monolithic forward pass took approximately 73.75 ms on average, of which `layer4` alone accounted for 30.70% (approximately 22.64 ms). More importantly, the computation remaining after a `layer4` split — average pooling plus the final linear layer — accounts for only about 0.32 ms in total, or roughly 0.44% of the full forward pass. This means that a split after `layer4` places approximately 99.6% of the total computation on the first service and leaves only about 0.4% on the second. By contrast, a split after `layer3` places approximately 68.8% of compute on the first service and 31.1% on the second, producing a substantially more balanced distribution.

The supporting timing analysis also shows that compute is concentrated in a small number of heavy internal units rather than being spread uniformly. As shown in table 5.7, the heaviest units are dominated by convolutional operations in later network regions, especially `layer4`. This makes a very late split structurally problematic: it transfers a small activation tensor, but it also leaves the remote side with almost no substantial computation.

This distinction between raw-fastest and structurally suitable is consistent with literature that treats compute distribution as a first-class partition criterion alongside communication cost [14, 19, 21, 22].

C. The RQ1.2 controlled comparison isolates the role of compute placement. The most precise evidence for the independent role of compute distribution comes from the RQ1.2 comparison between `split_after_layer3_block0` and `split_after_layer3`. Both conditions transfer an intermediate tensor of approximately 196 KB. If activation-transfer size were the sole determinant of boundary-crossing cost, these two conditions should exhibit similar boundary-crossing intervals and similar end-to-end latencies. They do not.

`split_after_layer3_block0` exhibited a boundary-crossing interval of 33.135 ms, while `split_after_layer3` exhibited a boundary-crossing interval of 25.718 ms — a difference of more than 7.4 ms despite identical activation size. The internal timing data helps explain this gap. The `layer3.1` block (the second residual block of `layer3`) accounts for approximately 9.95% of total monolithic compute. The difference is more directly explained by the amount of computation that remains on the second service. When the split is placed after `layer3.0`, Service B executes `layer3.1` in addition to `layer4`, pooling, and classification. When the split is placed after the full `layer3`, that computation remains on Service A. The internal timing report shows that `layer3.1` accounts for approximately 7.34 ms of monolithic compute, which closely matches the observed 7.42 ms difference in boundary-crossing interval between the two refined conditions. This provides direct evidence that the boundary-crossing difference is driven primarily by compute placement on the remote side rather than by activation-transfer size, which is identical in both cases.

This comparison also shows why a purely tensor-size-based explanation is incomplete. The two refined conditions share the same activation size, but the internal work performed after the boundary differs materially. In that sense, RQ1.2 isolates compute placement as the main explanatory variable within the carried-forward region.

This observation directly supports the principle established in the partitioning literature that compute distribution is not a secondary consideration but an independent factor in end-to-end split behavior and boundary-crossing cost [14, 19, 31].

D. Internal timing confirms that compute is unevenly distributed within ResNet-18. The internal timing experiment provides a structural explanation for why different boundaries within the same coarse region yield different performance. As visualised in fig. 5.2, the per-stage compute shares are approximately: stem 16.31%, `layer1` 17.72%, `layer2` 16.23%, `layer3` 18.56%, and `layer4` 30.70%. Convolution operations account for 81.8% of total compute, while batch normalisation contributes approximately 3.3% and operations below 0.1 ms (such as ReLU and residual addition) are individually negligible.

The operation-level support data adds a more concrete view of this distribution. As shown in table 5.7, the heaviest internal units are predominantly convolutional operations, with the largest shares concentrated in `layer4` and the late `layer3` region. By contrast, batch normalisation, residual addition, and ReLU operations are individually small and do not serve as meaningful compute anchors by themselves. This confirms that ResNet-18 is not only uneven at the stage level, but also internally dominated by a relatively small number of heavy convolutional units.

Two aspects of this distribution are relevant to the RQ1.1 and RQ1.2 findings. First,

layer4 is by far the most compute-intensive stage, which explains why a split after layer4 is structurally degenerate: it assigns the dominant compute block entirely to the first service. Second, the per-block decomposition within layer3 reveals that layer3.0 and layer3.1 do not contribute equally; the difference in their execution times contributes to the boundary-crossing asymmetry observed between split_after_layer3_block0 and split_after_layer3. This structural unevenness is consistent with the design of ResNet-18, where each successive stage doubles the channel count and halves the spatial resolution, producing progressively heavier convolution operations in later stages [12].

It should be noted that the internal timing experiment was conducted on the monolithic model and therefore measures local per-layer execution cost without the serialisation and communication overhead present in the split benchmarks. The timing values are used here to explain the compute-distribution dimension; the actual boundary-crossing cost is a composite of both communication and compute-related factors as measured in the split benchmarks themselves.

5.3.5 Interpretation

The explanatory analysis supports a coherent two-dimensional account of the local findings from RQ1.1 and RQ1.2.

At the coarse level (RQ1.1), activation-transfer size is the dominant factor explaining the gradient of boundary-crossing costs from early to late splits. The monotonic relationship between activation size and boundary-crossing interval is strong and consistent. However, this single dimension is insufficient: the compute-degenerate split_after_layer4 minimises transfer burden but fails as a meaningful decomposition because it collapses one side of the partition into a near-empty service.

At the refined level (RQ1.2), compute distribution emerges as an independent explanatory dimension. The controlled comparison between two conditions with identical activation size but different boundary-crossing intervals shows that the placement of computation around the boundary affects cost even when communication burden is held constant. The internal timing experiment supports this observation by confirming that per-stage and per-block compute within ResNet-18 is structurally uneven, with layer4 accounting for nearly a third of total forward-pass time and individual blocks within layer3 contributing unequally. The operation-level support data strengthens this explanation further by showing that the heaviest internal units are overwhelmingly convolutional and concentrated in the later part of the network.

Neither dimension alone is sufficient. Activation-transfer size explains the broad coarse pattern but not the refined within-region differences. Compute distribution explains the refined differences but does not account for the large boundary-crossing cost gradient between early and late coarse splits. Together, they provide a more complete explanation: suitable partition boundaries arise from a trade-off between activation-transfer burden, boundary-crossing cost, and compute distribution, not from the minimisation of any single metric.

5.3.6 Answer to RQ1.3

RQ1.3 shows that the latency and boundary-crossing behavior observed in RQ1.1 and RQ1.2 can be explained by two complementary structural properties of the partition: activation-transfer size and compute distribution.

Activation-transfer size is the dominant factor at the coarse level. The monotonic decrease in boundary-crossing interval from early to late splits in RQ1.1 closely tracks the reduction in intermediate tensor size as the boundary moves deeper into the network. This explains why early splits are costly and why later splits generally incur lower communication overhead.

Compute distribution is an independent and observable factor at the refined level. The RQ1.2 comparison between `split_after_layer3_block0` and `split_after_layer3` — which share the same activation size of approximately 196 KB yet differ in boundary-crossing interval by more than 7.4 ms — demonstrates that equal transfer burden does not guarantee equal performance. The placement of computation around the boundary matters independently of the size of the transferred tensor.

The internal timing experiment supports the compute-distribution explanation by confirming that execution cost within ResNet-18 is structurally uneven: convolution operations account for over 80% of total compute, `layer4` alone accounts for nearly 31%, and the heaviest internal units are concentrated in convolution-heavy late-stage regions. These structural properties explain both the compute degeneracy of the `layer4` split observed in RQ1.1 and the boundary-crossing asymmetry between the two 196 KB conditions observed in RQ1.2.

Taken together, under these controlled local conditions, suitable microservice boundaries for ResNet-18 arise from a joint trade-off among activation-transfer burden, boundary-crossing cost, and compute distribution across services [7, 16, 19, 33]. No single metric is sufficient. The mid-to-late coarse region around `layer2` and `layer3` remains the strongest carry-forward zone because it avoids both the large activation costs of early boundaries and the poor compute balance of overly late boundaries. These local explanatory findings provide the interpretive basis against which the later Kubernetes and Azure stages can evaluate whether infrastructure context changes the relative importance of these two dimensions.

5.4 RQ1.4 Multi-Microservice Kubernetes Inference Chain

5.4.1 Research Question

> *How does increasing the number of microservices affect end-to-end inference cost when ResNet-18 is decomposed into synchronously chained coarse architectural stages under in-cluster Kubernetes execution?*

RQ1.4 moves from the local split-selection stages of RQ1.1–RQ1.3 to the final thesis-facing local Kubernetes benchmark for the coarse-stage ResNet-18 chain family. Rather than selecting a boundary through carry-forward, this stage benchmarks a predefined set of five in-cluster conditions from `monolithic_k8s_1svc` to `chain_5svc` to measure how end-to-end latency changes as synchronous service boundaries are added under controlled local Kubernetes execution.

5.4.2 Literature Context

Prior work on split computing and distributed DNN inference consistently establishes that each additional service boundary introduces serialisation, transport, and coordination overhead. Neurosurgeon, JointDNN, and DeeperThings all report that partition quality depends jointly on compute placement and the activation transferred across each boundary, with early and spatially large feature maps making communication especially expensive [7, 16, 33]. Survey and optimisation-oriented work similarly frames boundary selection as a joint problem over transfer volume, compute balance, and deployment constraints rather than a simple function of network depth [19, 24, 35].

For multi-stage chains, this literature implies that end-to-end cost should generally rise as more boundaries are inserted, but that the size of each increment still depends on where those boundaries occur. ParetoPipe and DADS both show that pipeline latency is sensitive to boundary count and boundary placement, while hierarchical partitioning and containerised inference studies indicate that deployment context can add fixed coordination cost beyond raw model computation [8, 14, 21, 34]. RQ1.4 therefore evaluates a fixed coarse-stage chain family in Kubernetes and measures the resulting cost directly rather than assuming a uniform per-boundary penalty.

5.4.3 Experimental Setup

The thesis-facing RQ1.4 benchmark used the frozen controlled local Kubernetes run exported on 2026-04-21. The environment snapshot records an Ubuntu/Linux runtime (`Linux-6.17.0-22-generic-x86_64-with-glibc2.41`), Python 3.10.20, and PyTorch 2.11.0+cu130. Deployment metadata shows that the benchmark ran in namespace `rq14` on a single-node `kind` cluster (`kind-thesis-rq14`); for every condition, all service pods were scheduled on the same node, `thesis-rq14-control-plane`. The evaluated model was pretrained ResNet-18 with CPU-only FP32 execution and input shape $1 \times 3 \times 224 \times 224$.

CPU stabilisation controls were requested for the benchmark processes and applied where the runtime allowed. Threading controls for PyTorch, OpenMP, MKL, and OpenBLAS were all requested at one thread and observed at one thread. CPU affinity to one

Table 5.8: RQ1.4 benchmark configuration

Setting	Value
Runtime	Local Ubuntu/Linux Kubernetes benchmark
Cluster	Single-node `kind` cluster (`kind-thesis-rq14`)
Namespace	`rq14`
Conditions	Five predefined configurations from `monolithic_k8s_1svc` to `chain_5svc`
Execution mode	Local in-cluster rolling orchestration over the predefined condition set
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC (in-cluster Kubernetes)
gRPC port	50051
Maximum message size	16 MB
Service pod CPU request / limit	1 core / 1 core
Service pod memory request / limit	512 MiB / 512 MiB
Client CPU request / limit	1 core / 1 core
Client memory request / limit	1 GiB / 1 GiB
Image pull policy	`IfNotPresent`
Carry-forward	Not applicable; RQ1.4 benchmarks a predefined configuration set
Parity validation	Local segment validation and live gRPC chain validation; `num_inputs` $= 5$, `atol` $= 1 \times 10^{-5}$
Warmup calibration	Trailing window $= 10$, CV threshold $= 0.02$

physical core with SMT excluded was requested and applied; the observed affinity set was core 0. Process priority `nice -5` was requested and applied. Governor and turbo controls were also requested, but writes to `/sys` failed because the containerised environment exposed those paths as read-only. The detected governor was already `performance`, and turbo was already disabled. ASLR remained at the detected value 2 and was not modified.

Methodological validity was established before timing. For all five conditions, local segment validation against the monolithic model and live gRPC chain validation against

the deployed services both passed with `max_abs_diff` $= 0.0$, `atol` $= 1 \times 10^{-5}$, and `num_inputs` $= 5$.

5.4.4 Conditions

Five conditions were evaluated in a rolling orchestration sweep:

- **`monolithic_k8s_1svc`**: Full ResNet-18 inference in a single Kubernetes service. This is the Kubernetes-native monolithic baseline.
- **`chain_2svc`**: Two services with a split after `layer2`. Cumulative transferred activation per request: 980 KB.
- **`chain_3svc`**: Three services with splits after `layer1` and `layer3`. Cumulative transferred activation per request: 1568 KB.
- **`chain_4svc`**: Four services with splits after `layer1`, `layer2`, and `layer3`. Cumulative transferred activation per request: 1960 KB.
- **`chain_5svc`**: Five services with splits after `layer1`, `layer2`, `layer3`, and `layer4`. Cumulative transferred activation per request: 2058 KB.

Carry-forward is not applicable in RQ1.4. The frozen `carry_forward.json` states that carry-forward is defined only for the two-service split-selection experiments (RQ1.1 and RQ1.2), whereas RQ1.4 benchmarks this predefined Kubernetes chain set.

5.4.5 Results

Table 5.9: RQ1.4 summary of mean latency and overhead vs. Kubernetes monolithic baseline

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	81.447	79.648	4.061	89.444	—
chain_2svc	84.189	82.880	3.294	90.538	+2.742 / +3.4%
chain_3svc	87.608	85.487	4.274	96.083	+6.161 / +7.6%
chain_4svc	92.312	90.594	4.942	101.223	+10.865 / +13.3%
chain_5svc	92.624	90.312	4.318	99.398	+11.177 / +13.7%

Figure 5.3 visualises the same latency trend reported in table 5.9, showing that most of the increase occurs before the five-service configuration.

Table 5.10: RQ1.4 overhead and inferred non-compute/platform cost breakdown

Condition	Overhead (ms)	Overhead (%)	Cumul. Activation (KB)	Inferred Non-Compute / Pla
chain_2svc	2.742	3.4	980	6.285
chain_3svc	6.161	7.6	1568	9.646
chain_4svc	10.865	13.3	1960	12.231
chain_5svc	11.177	13.7	2058	13.893

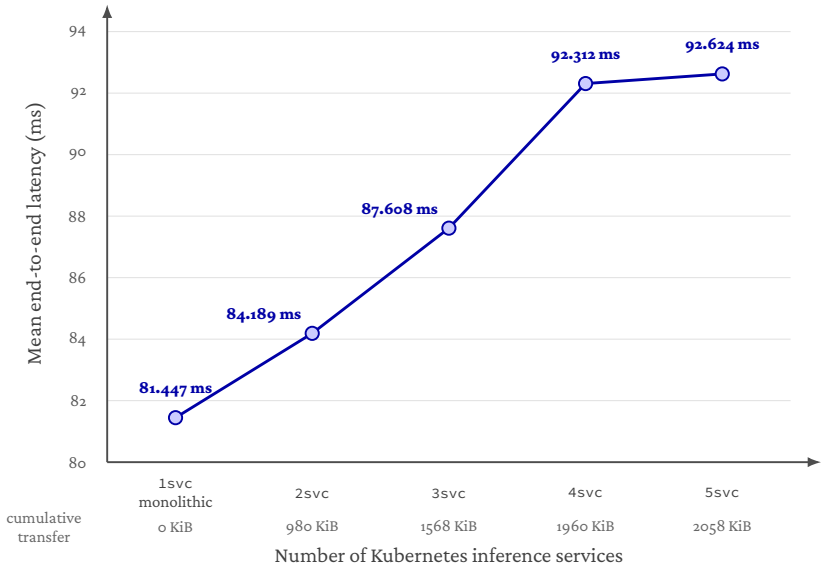


Figure 5.3: Mean end-to-end latency for the RQ1.4 local Kubernetes chain family. Latency increases as additional synchronous service boundaries are introduced, but the increase is not strictly linear. The smallest marginal increase occurs between the four-service and five-service chains, where the added late-stage boundary contributes comparatively little additional activation-transfer burden.

Table 5.11: RQ1.4 effect sizes relative to monolithic baseline

Condition	**Cohen's d**	**Interpretation**	**Mann-Whitney p**
`chain_2svc`	0.742	medium	2.38×10^{-81}
`chain_3svc`	1.478	large	6.82×10^{-150}
`chain_4svc`	2.402	large	3.03×10^{-282}
`chain_5svc`	2.667	large	3.50×10^{-291}

Methodological note. Effect sizes are computed from pooled per-iteration data ($N = 1{,}000$ iterations per condition). With large N, Mann-Whitney p-values are near-zero for any non-trivial difference and should not be over-interpreted as evidence of strong practical significance. Cohen's d provides a more informative measure of effect magnitude. Cross-round consistency and parity validation are the primary indicators of result stability and methodological validity.

The frozen controlled local Kubernetes run was stable across rounds. Round CVs remained low for all five conditions: 0.0162 for `monolithic_k8s_1svc`, 0.0141 for `chain_2svc`, 0.0163 for `chain_3svc`, 0.0174 for `chain_4svc`, and 0.0075 for `chain_5svc`. Together with the parity checks above, this supports treating the reported condition means as representative of the local in-cluster benchmark.

Table 5.12: RQ1.4 cross-round consistency

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
monolithic_k8s_1svc	81.447	1.3193	3.0408	0.0162
chain_2svc	84.189	1.1876	3.0546	0.0141
chain_3svc	87.608	1.4250	3.7465	0.0163
chain_4svc	92.312	1.6107	3.7851	0.0174
chain_5svc	92.624	0.6942	1.7056	0.0075

5.4.6 Interpretation

The results reveal four bounded findings for this controlled local Kubernetes benchmark.

Finding 1: Latency rises from one service to four services. Mean end-to-end latency increases from 81.447 ms for monolithic_k8s_1svc to 84.189 ms, 87.608 ms, and 92.312 ms for chain_2svc, chain_3svc, and chain_4svc, respectively. Relative to the Kubernetes monolithic baseline, the added overhead therefore grows from +2.742 ms (3.4%) to +10.865 ms (13.3%). Under this controlled local Kubernetes setup, the evaluated coarse-stage ResNet-18 family shows a clear accumulation of cost as synchronous service boundaries are added.

Finding 2: The increment from four to five services is small. The final step from chain_4svc to chain_5svc increases mean latency by only 0.312 ms, even though it adds one more boundary. This is consistent with the activation profile of the evaluated chain family: cumulative transferred activation rises from 1960 KB to 2058 KB, so the added late-stage boundary contributes only 98 KB. In this setup, later boundaries can therefore add less overhead when the additional transferred activation is small. This should be read as a property of the evaluated coarse-stage ResNet-18 chain family under this local benchmark, not as a universal law about microservice depth.

Finding 3: chain_2svc is the lowest-overhead non-monolithic condition. Among the chained deployments, chain_2svc remains the smallest non-monolithic overhead point at +2.742 ms (3.4%). Within the evaluated family, it is therefore the least expensive way to move from a single service to a chained deployment while preserving exact parity in the benchmarked workload. Because RQ1.4 uses a predefined condition set, this is an empirical summary of the final Kubernetes family rather than a carry-forward decision.

Finding 4: Scope of validity. Deployment metadata shows that all services ran on the same node of the local kind cluster. The result is therefore a valid local in-cluster Kubernetes benchmark, but it is still a single-node local cluster result. Broader cloud validation is deferred to RQ1.5.

5.4.7 Answer to RQ1.4

Under the final controlled local Kubernetes setup on the Ubuntu/Linux runtime, increasing the number of synchronously chained coarse-stage ResNet-18 services raises end-to-end latency from `monolithic_k8s_1svc` through `chain_4svc`, with mean latency increasing from 81.447 ms to 84.189 ms, 87.608 ms, and 92.312 ms. The smallest non-monolithic overhead is `chain_2svc` at +2.742 ms (3.4%), while `chain_5svc` reaches 92.624 ms, only 0.312 ms above `chain_4svc`. In this single-node local `kind` cluster, added boundaries therefore impose measurable but bounded overhead, and later boundaries can contribute less when the additional transferred activation is small. Carry-forward is not applicable in this stage because the Kubernetes condition set was predefined rather than selected.

5.5 RQ1.5 Azure Kubernetes Service Transfer Validation

5.5.1 Research Question

> *Does the local Kubernetes latency pattern observed for the coarse-stage ResNet-18 microservice chain transfer to Azure Kubernetes Service, and how do the absolute and relative costs change under managed cloud execution?*

RQ1.5 moves the predefined RQ1.4 chain family from local `kind` Kubernetes to Azure Kubernetes Service (AKS). The purpose is not to select a new decomposition, but to test whether the architectural ordering observed in local Kubernetes remains credible when the same synchronously chained inference workload is executed on managed cloud infrastructure. The stage therefore compares absolute latency, relative overhead, and condition ordering between the frozen RQ1.4 local Kubernetes run and the frozen RQ1.5 AKS run.

5.5.2 Literature Context

The split-computing literature predicts that the cost of a distributed DNN chain should depend jointly on the number and location of boundaries. Each boundary moves an intermediate activation through serialisation, transport, and deserialisation, while the size of the transferred tensor and the compute left on each side determine how costly that boundary becomes [7, 16, 33, 35]. Optimisation and pipeline-oriented studies likewise treat partition latency as a function of both communication and compute placement rather than service count alone [14, 19, 21]. This supports carrying the RQ1.4 chain family into AKS as a transfer-validation step: if the architecture matters more than local incidental noise, the relative ordering should remain interpretable even when the platform changes.

At the same time, microservice and cloud-systems work shows why absolute latency should not be expected to replicate exactly. RPC-based microservice calls pay a layered communication cost from protocol processing, serialisation, header parsing, memory copies, and network-stack traversal [2, 39]. Managed Kubernetes introduces further environment-specific factors. Prior work comparing managed Kubernetes services reports provider- and platform-dependent performance differences, while container, virtualisation, CNI, and cloud-noise studies show that VM type, networking path, host scheduling, and infrastructure variability can alter absolute measurements [6, 9, 10, 17]. For that reason, RQ1.5 treats preserved rank ordering and comparable overhead trends as directional transfer evidence, while avoiding the stronger claim that local Kubernetes and AKS should produce identical millisecond values [13].

5.5.3 Experimental Setup

The thesis-facing RQ1.5 benchmark used the frozen controlled AKS transfer-validation benchmark exported on 2026-04-22. The environment snapshot records a Linux Azure runtime (`Linux-5.15.0-1102-azure-x86_64-with-glibc2.41`), Python 3.10.20, and PyTorch 2.11.0+cu130. The deployment ran in namespace `rq15` on a single-node AKS cluster using Kubernetes 1.34, a `Standard_D8s_v3` node in the `swedencentral` Azure

region, and kubenet networking. Deployment metadata confirms that pod colocation was enforced: for every condition, the benchmark client and all service pods were scheduled on the same AKS node, aks-rq15pool-24384142-vmss000000. All AKS conditions used the same pinned container image digest: sha256:df3cf16f8760493993f8ad5ba103bc5508a130180baf5130ec44542a44513a8a.

Table 5.13: RQ1.5 AKS benchmark configuration

Setting	Value
Runtime	Azure Kubernetes Service benchmark
Cluster	Single-node AKS cluster, Kubernetes 1.34
Node type	Standard_D8s_v3
Region	swedencentral
Network plugin	kubenet
Namespace	rq15
Conditions	Five predefined configurations from monolithic_k8s_1svc to chain_5svc
Execution mode	In-cluster rolling orchestration with teardown between conditions
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC (in-cluster Kubernetes)
gRPC port	50051
Maximum message size	16 MB
Service pod CPU request / limit	1 core / 1 core
Service pod memory request / limit	1 GiB / 1 GiB
Client CPU request / limit	500m / 500m
Client memory request / limit	1 GiB / 1 GiB
Pod QoS	Guaranteed for service pods
Image pull policy	Always
Carry-forward	Not applicable; RQ1.5 evaluates a predefined chain family
Parity validation	Local segment validation and live gRPC chain validation; num_inputs $= 5$, atol $= 1 \times 10^{-5}$
Warmup calibration	Trailing window $= 10$, CV threshold $= 0.02$

CPU stabilisation controls were requested consistently with the preceding stages, and were applied where the managed AKS runtime allowed. Threading controls for PyTorch, OpenMP, MKL, and OpenBLAS were requested and observed at one thread. CPU affinity was applied to one physical core with SMT excluded; the observed affinity set was core 0. Process priority `nice -5` was requested but could not be applied due to insufficient privileges, so the benchmark ran at `nice=0`. CPU governor and turbo controls were unavailable through the exposed sysfs interface in AKS. These constraints are treated as realistic cloud-environment limitations of managed or virtualised execution, not as architectural effects of the microservice decomposition.

Before timing, both local chain-segment validation and live gRPC validation passed for every condition with `max_abs_diff` $= 0.0$ and `atol` $= 1 \times 10^{-5}$. This supports functional equivalence across the monolithic and chained AKS deployments.

5.5.4 Conditions

RQ1.5 reused the same predefined chain family as RQ1.4:

- **`monolithic_k8s_1svc`**: Full ResNet-18 inference in a single Kubernetes service.
- **`chain_2svc`**: Two services with a split after `layer2`. Cumulative transferred activation per request: 980 KB.
- **`chain_3svc`**: Three services with splits after `layer1` and `layer3`. Cumulative transferred activation per request: 1568 KB.
- **`chain_4svc`**: Four services with splits after `layer1`, `layer2`, and `layer3`. Cumulative transferred activation per request: 1960 KB.
- **`chain_5svc`**: Five services with splits after `layer1`, `layer2`, `layer3`, and `layer4`. Cumulative transferred activation per request: 2058 KB.

Carry-forward is not applicable in this stage. The experiment evaluates a fixed deployment family in AKS rather than choosing a new split point.

5.5.5 Results

Table 5.14: RQ1.5 AKS summary of mean latency and overhead vs. AKS monolithic baseline

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	65.619	65.312	1.291	67.693	—
chain_2svc	68.900	68.599	1.578	71.269	+3.280 / +5.0%
chain_3svc	71.884	71.488	1.701	74.798	+6.264 / +9.5%
chain_4svc	74.172	73.823	1.622	76.712	+8.553 / +13.0%
chain_5svc	75.695	75.339	1.826	77.993	+10.075 / +15.4%

Figure 5.4 visualises the transfer comparison in table 5.16. The overhead trend is similar across environments even though the absolute millisecond values differ.

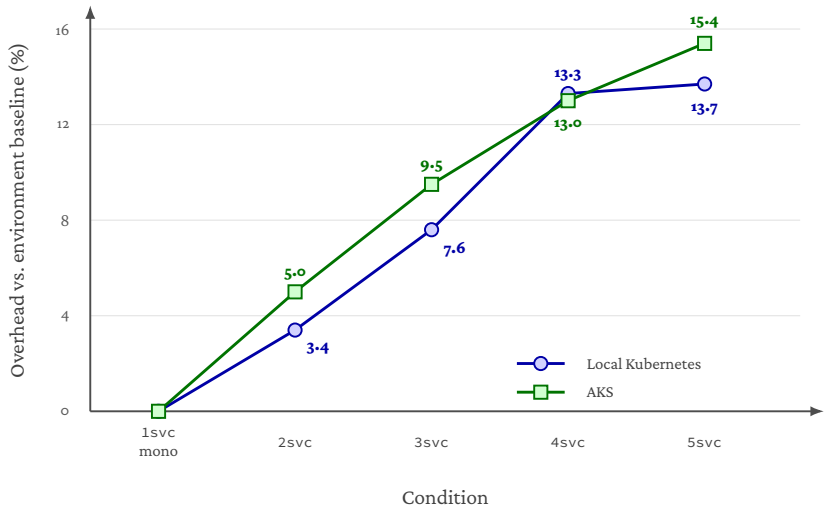
The rank ordering is identical between the frozen local Kubernetes and AKS runs: `monolithic_k8s_1svc`, `chain_2svc`, `chain_3svc`, `chain_4svc`, and `chain_5svc`.

Table 5.15: RQ1.5 AKS overhead and inferred non-compute/platform cost breakdown

Condition	Overhead (ms)	Overhead (%)	Cumul. Activation (KB)	Inferred Non-Compute / Platform (ms)
chain_2svc	3.280	5.0	980	4.053
chain_3svc	6.264	9.5	1568	6.337
chain_4svc	8.553	13.0	1960	8.041
chain_5svc	10.075	15.4	2058	9.381

Table 5.16: RQ1.5 transfer comparison between local Kubernetes and AKS

Condition	Local mean (ms)	AKS mean (ms)	Local overhead	AKS overhead
monolithic_k8s_1svc	81.447	65.619	0.0%	0.0%
chain_2svc	84.189	68.900	3.4%	5.0%
chain_3svc	87.608	71.884	7.6%	9.5%
chain_4svc	92.312	74.172	13.3%	13.0%
chain_5svc	92.624	75.695	13.7%	15.4%



Both environments preserve the same condition ordering; percentages are normalised to each environment's own monolithic baseline.

Figure 5.4: RQ1.5 normalised overhead transfer comparison between local Kubernetes and AKS. Overheads are computed relative to each environment's own monolithic Kubernetes baseline, so the figure visualises directional transfer of the service-count trend rather than raw latency equivalence.

Absolute latency is lower in the AKS run for every condition, but the within-environment ordering and the direction of the overhead trend are preserved.

The AKS run was highly stable across measured rounds. Round CVs ranged from 0.0017 to 0.0030, which is lower than the corresponding local Kubernetes CVs reported in RQ1.4. Warmup calibration indicated that each run reached a stable window at least once before measurement, while the cross-round CVs of the measured iterations provide the stronger stability evidence for the final reported means. As in the previous

Table 5.17: RQ1.5 AKS cross-round consistency

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
monolithic_k8s_1svc	65.619	0.1714	0.4432	0.0026
chain_2svc	68.900	0.1628	0.3750	0.0024
chain_3svc	71.884	0.1290	0.3199	0.0018
chain_4svc	74.172	0.2225	0.5384	0.0030
chain_5svc	75.695	0.1291	0.3284	0.0017

stage, effect sizes and Mann-Whitney tests were computed from pooled per-iteration data and are treated as supplementary because $N = 1{,}000$ per condition makes very small differences statistically significant. Cross-round stability, parity validation, and preserved ordering are the primary evidence for this stage.

5.5.6 Interpretation

The results reveal four bounded findings for the AKS transfer-validation stage.

Finding 1: AKS preserves the local Kubernetes condition ordering. The complete RQ1.4 ordering is preserved in AKS: the monolithic Kubernetes service is fastest, followed by chain_2svc, chain_3svc, chain_4svc, and chain_5svc. This provides directional transfer evidence. The same service-count trend that increased latency in local Kubernetes remains visible under managed cloud execution, even though the absolute millisecond values differ between environments.

Finding 2: AKS overhead is monotonic with service count. Relative to the AKS monolithic baseline, mean latency rises from 65.619 ms to 68.900 ms, 71.884 ms, 74.172 ms, and 75.695 ms as the chain grows from one to five services. The corresponding overheads are +3.280 ms (5.0%), +6.264 ms (9.5%), +8.553 ms (13.0%), and +10.075 ms (15.4%). Within this AKS deployment, adding synchronous service boundaries is therefore associated with measurable end-to-end cost.

Finding 3: Marginal overhead decreases toward later boundaries. The AKS marginal increments are +3.280 ms from the monolithic baseline to chain_2svc, +2.984 ms from chain_2svc to chain_3svc, +2.288 ms from chain_3svc to chain_4svc, and +1.523 ms from chain_4svc to chain_5svc. This is consistent with the bounded nonlinearity expected from the activation profile of the chain family: later added boundaries move smaller tensors, so an additional service boundary does not imply a constant per-boundary penalty.

Finding 4: Absolute latency is environment-specific. All AKS means are lower than their local Kubernetes counterparts, including the monolithic baseline. The lower AKS absolute latencies should not be interpreted as evidence that AKS is generally faster than local Kubernetes. The local and AKS runs differ in host CPU, kernel, virtualisation layer, Kubernetes implementation, container runtime context, CPU scheduling context,

and networking path. RQ1.5 therefore uses within-environment ordering and normalised overhead as the primary comparison, not direct absolute latency. On that basis, AKS provides directional transfer evidence for the RQ1.4 ordering while preserving the claim scope: absolute latency is deployment-specific, and the AKS run should not be interpreted as a numerical replication of the local Kubernetes measurements.

5.5.7 Answer to RQ1.5

Under the controlled AKS transfer-validation setup, the local Kubernetes service-count pattern transfers directionally to managed cloud execution. The AKS run preserves the complete RQ1.4 ordering: `monolithic_k8s_1svc` remains fastest at 65.619 ms, followed by `chain_2svc` at 68.900 ms, `chain_3svc` at 71.884 ms, `chain_4svc` at 74.172 ms, and `chain_5svc` at 75.695 ms. Relative overhead grows monotonically from 5.0% for `chain_2svc` to 15.4% for `chain_5svc`, while marginal increments shrink toward the final boundary. RQ1.5 therefore supports a directional transfer claim: the same architectural chain ordering remains visible in AKS, although absolute latency values are environment-specific and should not be interpreted as a direct replication of the local Kubernetes measurements.

5.6 RQ2.1 Service Identity and Mutual TLS Hardening

5.6.1 Research Question

> *What performance and operational overhead is introduced when the selected AKS microservice inference path is hardened with service identity, mutual TLS, and inter-service authorization policy?*

RQ2.1 adds the first security-hardening layer to the Azure deployment path selected from RQ1. The primary thesis-facing topology is `chain_2svc`, because it was the lowest-overhead non-monolithic chain in the preceding Kubernetes and AKS stages. The additional `chain_5svc` run is used as a stress case to examine whether the same security mechanism becomes more costly when the number of protected service boundaries increases. The goal is not to evaluate trusted execution environments in this stage, but to measure the cost of communication-level hardening: service identity, mutual TLS, and service-account-based authorization for inter-service traffic.

5.6.2 Literature Context

Service meshes are commonly used to add service discovery, observability, traffic policy, service identity, and mutual TLS to Kubernetes microservices without requiring application-level protocol changes. This makes them a natural first hardening layer for a distributed inference chain, because the inference services can remain application compatible while the mesh enforces identity-aware communication policies. Recent Kubernetes-security work similarly combines mutual TLS with policy-driven access control to constrain microservice workflows, supporting the decision to evaluate mTLS together with explicit authorization rather than encryption alone [4].

The same literature also warns that service-mesh hardening is not free. MeshInsight shows that sidecar-based meshes add latency and resource cost through extra IPC, kernel crossings, socket operations, buffer copies, and protocol parsing, and that the magnitude of the overhead depends strongly on the workload and mesh configuration [38]. mTLS-specific service-mesh benchmarks likewise report that enforcing mTLS through a mesh can materially affect latency, CPU, and memory, and that sidecar architecture and default proxy features may dominate the cost rather than the cryptographic primitive alone [5]. These findings provide the appropriate expectation for RQ2.1: the hardening layer should improve inter-service trust boundaries, but it should be judged by measured overhead in the target workload rather than by assuming a universal per-hop cost.

Finally, service identity and mTLS should not be overstated as a complete security solution. Work on service-mesh control-plane trust points out that a mesh CA and control plane remain security-critical components; compromising them can undermine the identity and policy assumptions of the data plane [30]. RQ2.1 therefore evaluates a bounded hardening claim: it measures the cost of adding mTLS and authorization policy to the AKS inference chain, while leaving stronger control-plane trust reduction and confidential-execution mechanisms to later protection layers.

5.6.3 Experimental Setup

The RQ2.1 benchmark used two frozen paired AKS artifact sets: a chain-2 main driver and a chain-5 maximum-depth stress driver. Both runs used the same AKS resource group and cluster name (`rg-thesis-rq15`, `thesis-rq15`), a single benchmark node in node pool `rq15pool`, and a separate system node pool for control-plane/system workloads. The benchmark node pool was tainted `workload=benchmark:NoSchedule`, and the effective configurations required strict same-node placement and control-plane isolation. Metadata confirms that the mesh control-plane pods did not share the benchmark node in the paired runs.

All RQ2.1 conditions used the same pinned container image digest, shown across two lines for typesetting:

```
sha256:524e8618c61d97f466b647cdc408e836
    256a94807790ad66321ae724f975a221
```

The mTLS condition used the managed AKS Istio add-on (`asm-1-29`). Each paired pass executed both security conditions, with the order alternated from a seeded plan: three passes ran mTLS first and two ran plain first. Each condition therefore has five executions and 1,000 measured iterations in total.

The plain condition used Kubernetes service-to-service communication without mesh injection. The mTLS condition added one Istio proxy per inference service, service accounts for the protected services, workload-level peer-authentication objects, and authorization-policy objects. The benchmark client remained outside the mesh. Consequently, RQ2.1 measures hardening of the inter-service path inside the inference chain rather than external ingress hardening. For `chain_2svc`, the downstream service was protected by strict mTLS and an authorization policy that allowed only the expected upstream service account. For `chain_5svc`, the same pattern was applied to each downstream segment, producing four immediate-upstream authorization policies.

Security validation passed in both frozen artifact sets. The mTLS runs passed static manifest validation and runtime validation, including the expected peer-authentication and authorization-policy counts. The full-chain positive probe passed for every mTLS pass, and direct non-mesh probes were blocked for the protected downstream segments. Resource metrics were complete in both runs.

5.6.4 Results

For the primary `chain_2svc` topology, the mTLS/AuthZ hardening bundle increased mean latency from 70.094 ms to 73.216 ms. The mean overhead was therefore 3.122 ms, or 4.5%, and p95 latency increased by 3.531 ms. The inferred non-compute/platform component increased from 4.268 ms to 7.284 ms. This supports the interpretation that most of the added latency is associated with the communication/platform path introduced by the mesh rather than with a change in neural-network computation.

For the additional `chain_5svc` stress topology, the same hardening bundle increased mean latency from 76.693 ms to 85.702 ms. The mean overhead was 9.008 ms, or 11.7%, and p95 latency increased by 10.028 ms. The inferred non-compute/platform component increased from 9.727 ms to 18.164 ms. The stress run therefore shows that the communication-

Table 5.18: RQ2.1 paired AKS security benchmark configuration

Setting	Value
Primary topology	`chain_2svc`; split after `layer2`
Stress topology	`chain_5svc`; splits after `layer1`, `layer2`, `layer3`, and `layer4`
Main artifact	`frozen_rq2_1_paired_20260429_102941_chain2`
Stress artifact	`frozen_rq2_1_paired_20260429_112307_chain5`
Conditions	Plain AKS chain vs. managed-Istio mTLS chain
Mesh revision	`asm-1-29`
Cluster	AKS `thesis-rq15`; isolated benchmark and system node pools
Benchmark node type	`Standard_D8s_v3`
Model	ResNet-18 (pretrained)
Device / precision	CPU / FP32
Input shape	$1 \times 3 \times 224 \times 224$
Paired passes	5
Warmup iterations	50 per condition execution
Measured iterations	200 per condition execution; 1,000 per condition after merging
Random seed	42
Service resources	1 CPU / 1 GiB per inference service
Client resources	100m CPU request, 1 CPU limit, 1 GiB memory
Sidecar resources	100m CPU request, 500m CPU limit, 128 MiB memory request, 512 MiB memory limit
Placement	Strict same-node service placement on tainted benchmark node pool
Security validation	Static manifest validation, runtime policy validation, full-chain positive probe, and non-mesh direct-denial probe

hardening cost grows when the number of protected service boundaries grows, even though the main RQ2.1 answer remains anchored in `chain_2svc`.

The paired-pass deltas in table 5.20 are consistent with the overall means. In `chain_2svc`, every paired pass shows a positive mTLS/AuthZ hardening overhead, with pass-level deltas between 2.693 ms and 3.318 ms. In `chain_5svc`, every paired pass is also positive, with deltas between 8.203 ms and 9.968 ms. The alternating execution order reduces the risk that the result is explained only by slow drift during the run.

Figure 5.5 visualises the paired latency deltas from table 5.20. The figure focuses on deltas rather than raw latency so that the two RQ2.1 findings remain visible: the selected two-service chain pays a modest overhead, while the deeper stress chain pays a larger overhead once four inter-service boundaries are protected.

Table 5.19: RQ2.1 latency results for plain and mTLS AKS chains

Topology	Condition	n	Mean (ms)	Median (ms)	p95 (ms)	Inferred non-compute/platform (ms)
chain_2svc	plain	1000	70.094	69.932	71.695	4.268
chain_2svc	mtls	1000	73.216	72.929	75.227	7.284
chain_5svc	plain	1000	76.693	76.235	79.918	9.727
chain_5svc	mtls	1000	85.702	85.112	89.947	18.164

Table 5.20: RQ2.1 paired-pass latency deltas

Topology	Mean delta (ms)	Mean delta (%)	Min–max pass delta (ms)	Pass-delta std (ms)	p95 delta (ms)
chain_2svc	3.122	4.5	2.693–3.318	0.259	3.531
chain_5svc	9.008	11.7	8.203–9.968	0.665	10.028

Deltas are computed as mTLS minus plain within the paired AKS benchmark design.

Figure 5.5: RQ2.1 mTLS/AuthZ latency overhead for the primary and stress AKS chains. Mean and p95 deltas are computed within the paired plain/mTLS design. The stress topology contains four protected inter-service boundaries, so the larger overhead visualises the depth sensitivity of sidecar-mediated communication hardening.

An additional chain_2svc ablation was run after the frozen paired benchmark to interpret the hardening bundle more precisely. This ablation is supporting evidence rather than a replacement for the frozen result, because it was collected in a separate campaign with a rebuilt image. Nevertheless, the full hardened ablation condition reproduced the frozen result's magnitude: C3–C0 added 3.045 ms, close to the frozen paired chain_2svc overhead of 3.122 ms. As shown in table 5.21, most of the adjacent latency change occurred when moving from plain Kubernetes to the mesh-default

Table 5.21: RQ2.1 chain-2 ablation of the communication-hardening bundle

Step	Condition	Mean latency (ms)	Adjacent delta (ms)	Interpretation
C0	Plain Kubernetes, no mesh	72.670	–	Non-mesh baseline
C1	Mesh-default sidecars / auto-mTLS / no AuthZ	75.533	+2.863	Main mesh data-path cost
C2	Strict downstream mTLS / no AuthZ	75.412	-0.122	Not distinguishable from pass noise
C3	Strict downstream mTLS plus AuthZ	75.715	+0.303	Small additional policy cost

condition. Strict mTLS enforcement over mesh-default traffic and the AuthorizationPolicy step were both small relative to pass-level variation. The correct interpretation is therefore not that pure TLS cryptography alone costs about 3 ms, but that the deployed managed-Istio data path introduces most of the measured overhead, while strict mTLS and identity-based authorization mainly add enforcement semantics with little additional steady-state latency in this workload.

Table 5.22: RQ2.1 resource and operational overheads

Topology	Sidecar CPU (mCPU)	Total pod CPU delta (mCPU)	Sidecar memory (MiB)	Total pod memory delta (MiB)	Schedule-to-ready delta (s)	Additional objects
chain_2svc	33.511	136.827	84.000	97.200	16.500	5
chain_5svc	82.141	72.481	222.686	267.429	19.280	14

The mTLS condition also increased resource requests and deployment complexity. For `chain_2svc`, the mesh added two injected proxies, two service accounts, two peer-authentication objects, one authorization-policy object, 200m requested sidecar CPU, and 256 MiB requested sidecar memory. For `chain_5svc`, the mesh added five injected proxies, five service accounts, five peer-authentication objects, four authorization-policy objects, 500m requested sidecar CPU, and 640 MiB requested sidecar memory. In both cases, three always-on Istio control-plane pods were present on the isolated system pool rather than the benchmark node. The CPU counter deltas should be read as window-level operational measurements rather than perfect causal attribution: in `chain_5svc`, measured application-container CPU was slightly lower in the mTLS condition, which makes the total pod CPU delta smaller than the sidecar CPU component itself.

5.6.5 Interpretation

The results support four bounded findings.

Finding 1: the mTLS/AuthZ hardening bundle adds measurable but bounded overhead to the selected chain. For the primary `chain_2svc` topology, the service-mesh hardening layer adds 3.122 ms, or 4.5%, to mean end-to-end latency. This is a measurable cost, but it is bounded relative to the approximately 70 ms plain AKS baseline. The result is consistent with service-mesh literature: sidecars add critical-path work, but the visible percentage overhead depends on the workload and on how much application compute the proxy cost is amortised over [38].

Finding 2: security overhead grows with protected chain depth. The `chain_5svc` stress run increases the mean hardening overhead to 9.008 ms (11.7%). This is nearly three times the absolute `chain_2svc` overhead. The larger cost is expected: the stress

topology contains more injected proxies, more authorization policies, and more protected inter-service boundaries. This finding reinforces the RQ1 result that service-count decisions and security decisions cannot be separated. A chain that is tolerable without communication hardening may become less attractive once every boundary also carries identity, encryption, proxying, and policy-enforcement cost.

Finding 3: the result is lower than some published Istio overheads, but not in conflict with them. The observed 4.5% primary overhead is much smaller than the high-load Istio penalties reported in broader service-mesh benchmarks [5]. This does not contradict that literature. Those studies use different services, protocols, load levels, tail-latency metrics, mesh configurations, and deployment environments. RQ2.1 uses a CPU-bound ResNet-18 inference chain with substantial per-request model computation, same-node placement, and a specific managed-Istio AKS configuration. The correct comparison is therefore qualitative: both this thesis and the literature show that sidecar-mediated mTLS has non-zero latency and resource cost, while the absolute magnitude is workload-specific.

Finding 4: RQ2.1 hardens inter-service communication, not the entire trust model. The security validation shows that the intended service-account-based communication policy was enforced for the protected downstream segments: valid full-chain requests succeeded, while direct non-mesh probes to protected downstream services were blocked. This is meaningful communication-level hardening. However, it does not remove trust in the service-mesh control plane, does not protect against all Kubernetes administrator threats, and does not provide confidential execution for model code or activations. Those limits matter because prior work identifies the mesh control plane and local CA as security-critical trust points [30]. RQ2.1 should therefore be claimed as a measured first hardening layer, not as a complete security solution.

5.6.6 Answer to RQ2.1

Under the paired AKS RQ2.1 benchmark, adding service identity, managed-Istio mTLS, and inter-service authorization policy introduces measurable but bounded overhead for the selected `chain_2svc` deployment. Mean latency increases from 70.094 ms in the plain condition to 73.216 ms with the mTLS/AuthZ hardening bundle, an overhead of 3.122 ms or 4.5%; p95 latency increases by 3.531 ms. Security validation passed in every mTLS pass, including full-chain positive probes and direct-denial probes against protected downstream segments. The additional `chain_5svc` stress run shows that the same hardening approach becomes more expensive as service depth increases: mean overhead rises to 9.008 ms or 11.7%. RQ2.1 therefore supports a cautious conclusion: service identity and managed-Istio mTLS/AuthZ are feasible for the selected Azure chain at a modest latency cost, but the cost is not negligible, grows with the number of protected service boundaries, and comes with operational complexity from sidecars, policies, resource requests, and readiness delay. The follow-up ablation indicates that this cost is dominated by entering the managed-Istio sidecar data path rather than by strict mTLS enforcement or AuthorizationPolicy evaluation alone.

5.7 RQ2.2 Confidential VM Execution Hardening

5.7.1 Research Question

> *What additional performance and operational overhead is introduced when VM-level confidential execution is added to the already communication-secured AKS inference chain?*

RQ2.2 adds a second security-hardening layer to the Azure deployment path. RQ2.1 already established the communication-secured baseline: the selected `chain_2svc` inference pipeline runs on AKS with managed-Istio mTLS, service identity, and an authorisation policy on the protected downstream service. RQ2.2 keeps that communication-security contract fixed and changes the execution environment of the inference services. The confidential condition runs selected services on Azure confidential VM node pools backed by AMD SEV-SNP, while the standard condition uses matched non-confidential AMD VM nodes.

The purpose of this stage is therefore incremental. It does not ask whether mTLS is useful; that was answered in RQ2.1. Instead, it asks what extra cost is paid when runtime protection against a stronger infrastructure threat model is added on top of the already secured service-to-service path.

5.7.2 Literature Context

AMD SEV-SNP is a VM-level confidential-computing mechanism. It extends the traditional virtualisation trust model by using hardware-supported memory encryption, integrity protection, and attestation so that a VM can be isolated from untrusted host software such as the hypervisor and cloud-management stack [3, 18]. Azure exposes this direction through confidential VMs and AKS confidential VM node pools, including AMD SEV-SNP confidential nodes that can be used as Kubernetes worker nodes [25, 26]. This matches the RQ2.2 threat-model extension: the protected inference service is no longer only protected in transit, but also runs inside a VM boundary intended to reduce exposure to host-level inspection or manipulation.

This protection boundary must be stated carefully. Virtualisation-based confidential computing protects the guest VM from parts of the surrounding platform, but it is not the same as application-level enclave isolation. The guest operating system, Python runtime, PyTorch process, service code, and Istio sidecar remain inside the VM trust boundary. If any of those components are compromised, SEV-SNP does not by itself isolate one model segment from the rest of the guest. Comparative work on virtualisation-based confidential-computing platforms also emphasises that these systems differ in their attacker models, attestation mechanisms, and residual side channels [11, 27]. AMD's own SEV-SNP documentation likewise treats several side-channel and availability threats as outside the core guarantee [3]. RQ2.2 therefore uses the term VM-level confidential execution rather than claiming complete application-level secrecy.

The interaction with the service mesh also matters. Istio mTLS protects traffic between mesh workloads, but in a sidecar-based deployment the proxy is responsible for TLS handling and the application still communicates with its local proxy inside the endpoint environment [15]. Consequently, plaintext exists inside the pod or VM boundary after

TLS termination. In the RQ2.2 confidential condition, the inference process and the sidecar for a confidentially placed service are both inside the confidential VM node boundary. This is useful for reducing exposure to the host platform, but it does not transform the mesh into end-to-end application-layer encryption.

Published performance results support measuring this overhead empirically rather than assuming a fixed cost. Confidential VM overheads are workload-dependent: CPU-bound workloads can see modest overhead, while memory-intensive, I/O-heavy, or virtualisation-sensitive workloads may pay more [27, 36]. Earlier work on general-purpose TEEs similarly shows that the overhead depends on the workload's memory, system-call, and I/O behaviour [1]. For machine learning specifically, confidential-computing surveys identify model confidentiality, input privacy, and protected execution as important motivations, while also noting that practical systems must account for performance, deployment complexity, and the chosen TEE boundary [28]. RQ2.2 is positioned within this trade-off: it measures the additional cost of running the already communication-secured inference chain on confidential VM infrastructure.

5.7.3 Experimental Setup

RQ2.2 used two frozen paired AKS artifact sets. The first run measured selective confidential execution, where only the downstream protected service was moved to an AMD SEV-SNP confidential node. The second run measured full two-service confidential execution, where both services in the chain were moved to AMD SEV-SNP confidential nodes. Both runs used the same application image within each paired artifact, preserved the RQ2.1 managed-Istio mTLS and authorisation semantics, and collected a newly paired standard baseline rather than comparing directly against old RQ2.1 data.

The selective run compared two conditions. In the standard condition, service1 and service2 both ran on `Standard_D8as_v5`. In the selective confidential condition, service1 remained on `Standard_D8as_v5`, while service2 ran on `Standard_DC8as_v5`. This condition isolates the cost of protecting the downstream service that receives intermediate activations and executes the later ResNet-18 segment.

The full-TEE run compared the same standard baseline pattern against a confidential condition in which both service1 and service2 ran on `Standard_DC8as_v5`. This extension measures the cost of placing the entire two-service inference chain inside confidential VM infrastructure while still preserving the RQ2.1 mTLS/AuthZ service-communication contract.

5.7.4 Results

In the selective service2-only TEE run, mean latency increased from 60.204 ms to 62.263 ms. The mean overhead was therefore 2.059 ms, or 3.420%. Median latency increased by 1.831 ms, and p95 latency increased by 3.850 ms. Sequential throughput, computed from the mean latency, decreased from 16.610 to 16.061 requests per second, a reduction of 3.307%.

In the full two-service TEE run, mean latency increased from 60.126 ms to 65.031 ms. The mean overhead was 4.904 ms, or 8.156%. Median latency increased by 4.392 ms,

Table 5.23: RQ2.2 benchmark configuration

Setting	Value
Topology	`chain_2svc`; split after `layer2`
Communication security	Managed-Istio mTLS, service identity, and authorisation policy inherited from RQ2.1
Standard VM size	`Standard_D8as_v5`
Confidential VM size	`Standard_DC8as_v5` with AMD SEV-SNP
Region	`westeurope`
Model	ResNet-18 (pretrained)
Device / precision	CPU / FP32
Input shape	$1 \times 3 \times 224 \times 224$
Paired passes	5 per artifact
Warmup iterations	50 per condition execution
Measured iterations	200 per condition execution; 1,000 per condition after merging
Selective TEE artifact	`frozen_rq2_2_confidential_20260502_200835_1_tee`
Full TEE artifact	`frozen_rq2_2_confidential_20260503_085320_full_tee`

Table 5.24: RQ2.2 selective service2-only confidential execution results

Condition	Service2 VM	n	Mean (ms)	Median (ms)	p95 (ms)
`standard`	`Standard_D8as_v5`	1000	60.204	59.631	63.982
`confidential`	`Standard_DC8as_v5`	1000	62.263	61.462	67.832
Delta	—	—	**+2.059 / +3.420%**	**+1.831 / +3.071%**	**+3.850 / +6.017%**

Table 5.25: RQ2.2 full two-service confidential execution results

Condition	Service1 VM	Service2 VM	n	Mean (ms)	Median (ms)	p95 (ms)
`standard`	`Standard_D8as_v5`	`Standard_D8as_v5`	1000	60.126	59.766	63.040
`confidential`	`Standard_DC8as_v5`	`Standard_DC8as_v5`	1000	65.031	64.158	68.250
Delta	—	—	—	**+4.904 / +8.156%**	**+4.392 / +7.349%**	**+5.209 / +8.264%**

and p95 latency increased by 5.209 ms. Sequential throughput decreased from 16.632 to 15.377 requests per second, a reduction of 7.541%.

Table 5.26: RQ2.2 selective versus full confidential execution overhead

TEE scope	Mean delta	Median delta	p95 delta	Throughput delta
Service2 only	+2.059 ms / +3.420%	+1.831 ms / +3.071%	+3.850 ms / +6.017%	-0.549 req/s / -3.307%
Full two-service chain	+4.904 ms / +8.156%	+4.392 ms / +7.349%	+5.209 ms / +8.264%	-1.254 req/s / -7.541%

Table 5.26 shows the main trade-off. Moving only the downstream protected service to confidential infrastructure adds a smaller mean-latency cost than moving both services. Full-chain confidential execution roughly doubles the absolute mean over-

head compared with service2-only hardening, from 2.059 ms to 4.904 ms. However, it also expands the confidential-computing boundary from the downstream segment to the full two-service inference chain.

5.7.5 Interpretation

The results support four bounded findings.

Finding 1: VM-level confidential execution adds measurable overhead on top of mTLS. Both RQ2.2 paired runs show positive confidential-execution overhead relative to their newly collected standard baselines. This matters because the baselines already include the RQ2.1 mTLS/AuthZ hardening. The measured deltas therefore represent the additional cost of changing the execution environment to confidential VM infrastructure, not the cost of introducing service-mesh communication security.

Finding 2: selective TEE is the lower-cost hardening point. The service2-only condition increases mean latency by 3.420%. This is the most direct implementation of the original RQ2.2 design: service2 is the downstream protected component, receives intermediate activations from service1, and is the service guarded by the strict mTLS and authorisation policy in the RQ2.1 contract. Selectively placing that service on AMD SEV-SNP infrastructure gives the thesis a narrow, defensible hardening point with a modest measured cost.

Finding 3: full-chain TEE strengthens coverage but increases cost. The full two-service condition increases mean latency by 8.156%, compared with 3.420% for service2-only TEE. This result is consistent with the expected workload-dependent cost of confidential VMs: extending the confidential boundary to more runtime components places more of the critical path on confidential VM infrastructure [27, 36]. The full-chain run is therefore useful for RQ2.3, because it shows that stronger coverage is available but is not free.

Finding 4: the security claim is stronger than RQ2.1 but still bounded. RQ2.1 protected the service-to-service communication path. RQ2.2 adds a VM-level runtime protection boundary against host-level access for the confidentially placed services. This is a stronger security posture, but it is not complete application-level isolation. The guest OS, application process, model runtime, and local sidecar remain trusted parts of the confidential VM environment. Plaintext also exists inside the endpoint boundary after mTLS termination. The correct claim is therefore that RQ2.2 adds confidential VM execution to the already communication-secured Azure inference chain, not that it removes every trusted component or eliminates all possible data exposure.

5.7.6 Answer to RQ2.2

RQ2.2 shows that AMD SEV-SNP VM-level confidential execution is feasible for the communication-secured AKS inference chain and introduces measurable but bounded additional overhead. When only the protected downstream service is moved to a confidential node,

mean latency increases by 2.059 ms, or 3.420%, and p95 latency increases by 3.850 ms. When both services in the two-service chain are moved to confidential nodes, mean latency increases by 4.904 ms, or 8.156%, and p95 latency increases by 5.209 ms.

These results support a clear trade-off: selective TEE provides the lower-overhead confidential-execution option for the sensitive downstream segment, while full-chain TEE provides broader VM-level runtime protection at a higher performance cost. In both cases, the interpretation remains bounded by the SEV-SNP threat model and by the sidecar-based service-mesh architecture: RQ2.2 evaluates VM-level confidential execution on Azure, not application-level enclave isolation or complete end-to-end secrecy.

5.8 RQ2.3 Security-Hardening Trade-off Synthesis

5.8.1 Research Question

This subsection addresses the following sub-research question:

> Which security-hardening configuration provides the most appropriate trade-off between performance cost, operational complexity, and protection scope for the selected AKS microservice inference deployment?

RQ2.3 is a synthesis question rather than a new benchmark. RQ2.1 measured the cost of adding service identity, mutual TLS, and inter-service authorization policy to the selected Azure inference path. RQ2.2 then measured the additional cost of placing one or both inference services on AMD SEV-SNP confidential VM node pools while keeping the communication-security contract fixed. RQ2.3 combines these results to compare the evaluated hardening levels and identify when each level is appropriate.

5.8.2 Synthesis Basis

The synthesis compares four deployment configurations. The first is the plain AKS baseline from RQ1.5, where the inference chain runs without service-mesh mTLS or confidential-node placement. The second is the RQ2.1 communication-secured configuration, where managed-Istio mTLS, service identity, and authorization policy are added to the selected chain. The third is the RQ2.2 selective confidential-execution configuration, where only the downstream protected service runs on an AMD SEV-SNP confidential node. The fourth is the RQ2.2 full two-service confidential-execution configuration, where both inference services run on confidential nodes.

These configurations should not be interpreted as interchangeable variants of the same security mechanism. They protect different parts of the system. Plain AKS provides the performance baseline but does not add the communication-level identity and encryption policy evaluated in RQ2.1. The mTLS/AuthZ configuration hardens the service-to-service path by authenticating workloads and enforcing authorization policy. The confidential-node configurations add a VM-level runtime protection boundary for the services placed on AMD SEV-SNP infrastructure. The selective TEE configuration protects only the downstream service, while the full TEE configuration expands that confidential-computing boundary to both services.

Table 5.27: RQ2.3 synthesis of evaluated security-hardening levels.

Configuration	Protection scope	Measured cost	Thesis interpretation
Plain AKS	Baseline AKS deployment without mesh mTLS/AuthZ or confidential-node placement.	Fastest evaluated Azure deployment path.	Best latency baseline, but weakest protection in the evaluated security set. Suitable only when additional communication and runtime hardening are not required.
mTLS/AuthZ	Service identity, encrypted service-to-service communication, and authorization policy for the selected inference path.	chain_2svc: +3.122 ms / +4.5%; chain_5svc stress case: +9.008 ms / +11.7%.	Best general-purpose hardening layer when the main requirement is authenticated and encrypted inter-service communication. Cost is measurable but bounded for the selected chain, and grows with protected chain depth.
mTLS/AuthZ + selective TEE	Communication hardening plus AMD SEV-SNP VM-level confidential execution for the downstream protected service.	+2.059 ms / +3.420% over the newly paired standard-node mTLS baseline.	Best trade-off when one remote or downstream segment is the main sensitivity boundary. Adds runtime protection at lower cost than full two-service confidential execution.
mTLS/AuthZ + full two-service TEE	Communication hardening plus AMD SEV-SNP VM-level confidential execution for both inference services.	+4.904 ms / +8.156% over the newly paired standard-node mTLS baseline.	Broadest runtime-protection scope evaluated in this thesis, but with higher latency and throughput cost than selective TEE. Suitable when both services are considered sensitive.

5.8.3 Trade-off Summary

The trade-off is not a single global ranking because the preferred configuration depends on the threat model. If performance is the only criterion, the plain AKS deployment is the best option. However, it does not provide the additional service identity, encrypted inter-service communication, or authorization-policy enforcement evaluated in RQ2.1. If the main security requirement is to protect communication between inference services, the mTLS/AuthZ configuration provides the most appropriate default balance. It adds a modest mean latency overhead of 3.122 ms, or 4.5%, for the selected chain_2svc topology, while enforcing the intended service-account-based policy. The chain_5svc stress case shows that this cost grows with the number of protected boundaries, reaching 9.008 ms, or 11.7%, when the full five-service chain is protected.

If the threat model includes host-level exposure of a selected inference component, mTLS alone is not sufficient because it protects the communication path rather than the runtime memory of the service. In that case, selective confidential execution provides

a narrower and lower-cost hardening step. The service2-only confidential-node run increases mean latency by 2.059 ms, or 3.420%, relative to its paired standard-node mTLS baseline. This makes selective TEE the most attractive configuration when the downstream service is the main sensitivity boundary, for example because it receives intermediate activations and executes the later model segment.

The full two-service confidential configuration extends the protection scope further. Both inference services run on AMD SEV-SNP confidential nodes, so the VM-level runtime protection boundary covers the complete two-service inference chain. This stronger coverage is not free: mean latency increases by 4.904 ms, or 8.156%, and throughput decreases by 7.541% relative to the paired standard-node mTLS baseline. The full-TEE configuration is therefore the strongest evaluated runtime-hardening option, but it is appropriate only when the broader protection scope justifies the additional performance cost.

5.8.4 Interpretation

RQ2.3 shows that the evaluated security mechanisms form a cumulative hardening gradient rather than a single binary secure/insecure choice. The RQ1.5 plain AKS deployment establishes the performance baseline. RQ2.1 adds communication-level security through mTLS, service identity, and authorization policy. RQ2.2 then adds VM-level confidential execution for either one selected service or the full two-service chain. Each step increases the protection scope, but each step also introduces additional latency, resource, or operational cost.

The main engineering implication is that security hardening should be scoped to the threat model. Applying every available mechanism everywhere is not always the best default. In the evaluated deployment, mTLS/AuthZ is the most suitable general-purpose security baseline because it protects the inter-service path at a modest cost for chain_2svc. Selective TEE is the best next step when the downstream segment is the primary sensitivity boundary, because it adds confidential-node execution with lower overhead than full-chain TEE. Full-chain TEE gives broader runtime protection but should be treated as a stronger, more expensive configuration rather than as the default choice for all deployments.

The results also show that performance and security cannot be evaluated independently. RQ1 established that service boundaries add measurable overhead and that this overhead depends on boundary placement and service count. RQ2 shows that security mechanisms compound this cost. The mTLS/AuthZ hardening overhead grows when more service boundaries are protected, and confidential-execution overhead grows when the confidential-computing scope expands from one service to both services. Therefore, the final deployment decision depends on three linked factors: the chosen model partition, the number of service boundaries, and the required protection scope.

5.8.5 Answer to RQ2.3

RQ2.3 shows that the best hardening level depends on the assumed threat model and the acceptable performance cost. Plain AKS is the fastest evaluated configuration, but it provides the weakest protection in the security comparison. mTLS with service identity

and authorization policy is the best general-purpose balance when the main goal is authenticated and encrypted service-to-service communication; in the selected chain_2svc deployment it adds 3.122 ms, or 4.5%, mean latency overhead. Selective TEE is the preferred option when one downstream inference segment is the main sensitivity boundary, adding 2.059 ms, or 3.420%, over its paired mTLS standard-node baseline. Full two-service TEE provides the broadest VM-level runtime protection evaluated in this thesis, but increases mean latency by 4.904 ms, or 8.156%, and therefore represents a stronger but more expensive hardening point.

Overall, the evaluated configurations support a cumulative security-performance trade-off: stronger protection is feasible for the selected AKS inference chain, but each additional protection layer adds measurable cost. The most defensible default is mTLS/AuthZ for communication-level protection, with selective or full confidential execution added only when the threat model requires runtime protection against host-level exposure.

Chapter 6

Analysis

This chapter synthesises the empirical results across the staged pipeline. The individual RQ chapters report the detailed measurements; the purpose here is to connect those results into a single interpretation of what happens when ResNet-18 inference is decomposed into microservices, moved through Kubernetes and AKS, and then hardened with communication-level and VM-level security mechanisms.

6.1 Architectural Split Choice

The local split-selection stages show that the position of the model boundary matters before Kubernetes or cloud effects are introduced. RQ1.1 found that coarse residual-stage boundaries are not interchangeable: early boundaries carry high activation-transfer cost, while the very late `layer4` boundary becomes compute-degenerate. The strongest carry-forward region is therefore the mid-to-late part of the network around `layer2` and `layer3`, as shown in table 5.2. This matches the broader split-computing literature, which treats partitioning as a trade-off among activation size, compute allocation, and deployment conditions [16, 19, 33, 35].

RQ1.2 refined this region and showed that a finer boundary can change behavior even when activation size is held constant. The comparison between `split_after_layer3_block0` and `split_after_layer3` is the important case: both transfer the same approximate activation size, but their boundary-crossing intervals differ. This supports the RQ1.3 interpretation that activation-transfer burden and compute placement are complementary explanations rather than competing ones [21, 31, 34].

6.2 Deployment Context

RQ1.4 and RQ1.5 show that the architectural pattern remains visible after the workload is moved into Kubernetes-style service chains. In local `kind`, latency increases from the monolithic Kubernetes baseline through the four-service chain, while the final five-service increment is small because the added late-stage boundary transfers a much smaller activation tensor. In AKS, the same condition ordering is preserved: `monolithic_k8s_1svc`, `chain_2svc`, `chain_3svc`, `chain_4svc`, and `chain_5svc`. The absolute millisecond

values differ between local Kubernetes and AKS, but the within-environment overhead trend transfers directionally, as summarised in table 5.16.

This distinction is important. The thesis does not claim that local Kubernetes predicts AKS latency numerically. Cloud and Kubernetes performance studies show that container runtime, virtualization, CNI, managed-cluster implementation, and cloud variability can change absolute measurements [6, 9, 10, 17]. The stronger supported claim is architectural: when the same ResNet-18 chain family is run in AKS, the relative ordering remains interpretable even though the platform changes.

6.3 Security Hardening

The security stages show that hardening is feasible for the selected AKS deployment, but it is not free. RQ2.1 adds managed-Istio mTLS, service identity, and authorization policy. For the selected `chain_2svc` topology, the measured mean overhead is 3.122 ms, or 4.5%; for the `chain_5svc` stress topology, the overhead grows to 9.008 ms, or 11.7% (tables 5.19 and 5.20). This is consistent with service-mesh studies showing that sidecar-based mTLS adds latency and resource cost through additional critical-path processing [5, 38].

RQ2.2 adds VM-level confidential execution on AMD SEV-SNP nodes while keeping the mTLS contract fixed. Selective confidential execution for only the downstream service adds 2.059 ms, or 3.420%, over its paired standard-node mTLS baseline. Full two-service confidential execution adds 4.904 ms, or 8.156%, over its paired standard-node mTLS baseline. These results match the confidential-computing literature's expectation that overhead is workload- and scope-dependent [27, 28, 36].

6.4 Overall Interpretation

The main engineering lesson is that partitioning, deployment, and hardening cannot be optimised independently. The split point determines activation-transfer size and compute placement. Kubernetes and AKS add platform-specific communication and scheduling effects. Security hardening then compounds the cost of the selected service topology. This means that a decomposition that is acceptable without mTLS or confidential execution may become less attractive once every service boundary is protected.

For the evaluated workload, `chain_2svc` is the most defensible non-monolithic deployment point. It avoids the excessive boundary count of deeper chains, preserves a meaningful architectural split, and remains the lowest-overhead chained condition in both local Kubernetes and AKS. It also provides the practical base for RQ2: mTLS/AuthZ is measurable but bounded on this topology, and selective TEE can protect the downstream service at a lower cost than full-chain confidential execution.

6.5 Threats to Validity

The strongest validity support is internal consistency. The experiments use fixed model weights, fixed input shape, functional parity validation, repeated rounds, and a shared

benchmark contract. The local and Kubernetes stages also use CPU-affinity and single-threading controls where the runtime permits. These choices follow systems benchmarking guidance that stresses controlled comparisons, explicit measurement context, and caution when interpreting performance numbers [13].

The main limitation is external validity. The workload is one ResNet-18 inference pipeline, the request pattern is sequential batch-size-one inference, and the Kubernetes and AKS runs use same-node placement. Multi-node clusters, concurrent clients, GPUs, larger models, feature compression, autoscaling, and different service-mesh configurations could change the absolute overheads and possibly the preferred topology. The thesis therefore supports bounded claims about the evaluated system rather than a universal rule for all microservice inference deployments.

Chapter 7

Conclusion

7.1 Summary & Findings

This thesis evaluated ResNet-18 inference as a staged microservice system. The study started with controlled local split selection, moved the selected chain family into local Kubernetes and AKS, and then measured the additional cost of service-mesh mTLS, authorization policy, and AMD SEV-SNP confidential VM execution.

The main finding is that microservice inference cost is shaped by three linked factors: where the neural network is split, where the resulting services are deployed, and how strongly the communication and execution path are hardened. RQ1 showed that suitable ResNet-18 boundaries lie in the mid-to-late region around `layer2` and `layer3`, because early boundaries transfer larger activations and overly late boundaries become compute-degenerate. RQ1.4 and RQ1.5 then showed that the same service-count ordering remains visible in Kubernetes and AKS, although absolute latency is environment-specific. RQ2 showed that mTLS/AuthZ and confidential VM execution are feasible for the selected AKS chain, but each protection layer adds measurable overhead.

7.2 Contributions

The thesis makes three contributions. First, it provides a staged empirical evaluation method for neural-network microservice decomposition, moving from local split selection to Kubernetes, AKS, and security hardening without changing the model or benchmark contract. Second, it shows that activation-transfer size and compute placement must be considered together when selecting ResNet-18 service boundaries. Third, it quantifies a security-performance trade-off for the selected Azure deployment path: mTLS/AuthZ is a practical communication-hardening baseline, selective TEE protects the downstream service at lower cost, and full-chain TEE broadens VM-level runtime protection at higher latency cost.

7.3 Limitations & Threats to Validity

The conclusions are bounded by the evaluated workload and deployment setup. The thesis uses ResNet-18, CPU-only FP32 inference, batch size one, synchronous gRPC calls, and same-node Kubernetes and AKS placement. The results should therefore not be generalized directly to GPU inference, concurrent serving, multi-node clusters, larger models, feature-compressed split computing, or different service-mesh configurations.

The benchmark design reduces internal validity risks through repeated rounds, fixed inputs and weights, functional parity checks, and a shared measurement contract. Still, cloud measurements remain sensitive to platform conditions, and the AKS and confidential VM results should be read as measured evidence for this deployment path rather than as provider-independent constants.

7.4 Future Work

Future work should extend the evaluation to larger and more diverse models, including transformer-based inference workloads and models with different activation profiles. The same staged method could also be applied under concurrent load, GPU execution, multi-node Kubernetes placement, and autoscaling conditions. Another useful extension is to combine architectural partitioning with feature compression, since compression may change the cost of early split points. Finally, the security evaluation could be expanded to compare sidecar and sidecarless meshes, application-layer encryption, attestation workflow integration, and different confidential-computing platforms.

Appendix A

Appendix

In the appendix you can provide non-crucial information. Often you put here tables or figures that are too big and do not contribute to the results or evaluation chapter. Same with longer code examples, or design files.

References

[1] A. Akram, A. Giannakou, V. Akella, J. Lowe-Power and S. Peisert. "Performance Analysis of Scientific Computing Workloads on General Purpose TEEs". In: *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2021, pp. 1066–1076. doi: 10.1109/IPDPS49936.2021.00115.

[2] P. Alvaro, M. Adiletta, A. Cockcroft, F. Hady, R. Illikkal, E. Ramos, J. Tsai and R. Soulé. "NotNets: Accelerating Microservices by Bypassing the Network". In: *Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '24)*. ACM, 2024. doi: 10.1145/3678015.3680494.

[3] AMD. *AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More*. Tech. rep. Advanced Micro Devices, Inc., 2020. url: https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.

[4] B. Ashraf, S. Kumar and N. Jawad. "A Policy-Driven Approach for Securing Microservices Workflow in Kubernetes Cluster". In: *2025 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*. IEEE, 2025. doi: 10.1109/CLUSTERWorkshops65972.2025.11164216.

[5] A. Bremler Barr, O. Lavi, Y. Naor, S. Rampal and J. Tavori. "Performance Comparison of Service Mesh Frameworks: the MTLS Test Case". In: *2025 IEEE/IFIP Network Operations and Management Symposium (NOMS)*. IEEE, 2025. doi: 10.1109/NOMS57970.2025.11073712.

[6] D. De Sensi, T. De Matteis, K. Taranov, S. Di Girolamo, T. Rahn and T. Hoefler. "Noise in the Clouds: Influence of Network Performance Variability on Application Scalability". In: *ACM Transactions on Parallel Computing* (2023). doi: 10.1145/3570609.

[7] A. E. Eshratifar, M. S. Abrishami and M. Pedram. "JointDNN: An Efficient Training and Inference Engine for Intelligent Mobile Cloud Computing Services". In: *IEEE Transactions on Mobile Computing* 20.2 (2021), pp. 565–576. doi: 10.1109/TMC.2019.2947893.

[8] "Evaluating Kubernetes Performance for GenAI Inference". In: *arXiv preprint arXiv:2602.04900* (2026). doi: 10.48550/arXiv.2602.04900. url: https://arxiv.org/abs/2602.04900.

[9] A. P. Ferreira and R. O. Sinnott. "A Performance Evaluation of Containers Running on Managed Kubernetes Services". In: *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 2019, pp. 199–208. doi: 10.1109/CloudCom.2019.00049.

[10] S. Giallorenzo, J. Mauro, M. G. Poulsen and F. Siroky. "Virtualization Costs: Benchmarking Containers and Virtual Machines Against Bare-Metal". In: *SN Computer Science* 2.4 (2021), p. 404. doi: 10.1007/s42979-021-00781-8.

[11] R. Guanciale, N. Paladi and A. Vahidi. "SoK: Confidential Quartet – Comparison of Platforms for Virtualization-Based Confidential Computing". In: *2022 IEEE International Symposium on Secure and Private Execution Environment Design (SEED)*. IEEE, 2022, pp. 109–120. doi: 10.1109/SEED55351.2022.00017.

[12] K. He, X. Zhang, S. Ren and J. Sun. "Deep Residual Learning for Image Recognition". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[13] T. Hoefler and R. Belli. "Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015. doi: 10.1145/2807591.2807644.

[14] C. Hu, W. Bao, D. Wang and F. Liu. "Dynamic Adaptive DNN Surgery for Inference Acceleration on the Edge". In: *IEEE INFOCOM 2019 — IEEE Conference on Computer Communications*. IEEE, 2019. doi: 10.1109/INFOCOM.2019.8737614.

[15] Istio. *Understanding TLS Configuration*. Istio Documentation. Accessed: 2026-05-03. 2023. url: https://istio.io/latest/docs/ops/configuration/traffic-management/tls-configuration/.

[16] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars and L. Tang. "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge". In: *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2017, pp. 615–629. doi: 10.1145/3037697.3037698.

[17] Z. Kang, K. An, A. Gokhale and P. Pazandak. "A Comprehensive Performance Evaluation of Different Kubernetes CNI Plugins for Edge-based and Containerized Publish/Subscribe Applications". In: *2021 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2021. doi: 10.1109/IC2E52221.2021.00017.

[18] D. Kaplan. "Hardware VM Isolation in the Cloud: Enabling Confidential Computing with AMD SEV-SNP Technology". In: *Communications of the ACM* 67.1 (2023), pp. 54–59. doi: 10.1145/3623392.

[19] P. Kayal and A. Leon-Garcia. "DNNSplit: Latency and Cost-Efficient Split Point Identification for Multi-Tier DNN Partitioning". In: *IEEE Access* 12 (2024), pp. 79895–79911. doi: 10.1109/ACCESS.2024.3409057.

[20] J. Li, W. Liang, Y. Li, Z. Xu and X. Jia. "Delay-Aware DNN Inference Throughput Maximization in Edge Computing via Jointly Exploring Partitioning and Parallelism". In: *2021 IEEE 46th Conference on Local Computer Networks (LCN)*. 2021. doi: 10.1109/LCN52139.2021.9524928.

[21] A. Masud, N. Foley, P. D. Rajarajan and P. Lama. "Where to Split? A Pareto-Front Analysis of DNN Partitioning for Edge Inference". In: *arXiv preprint arXiv:2601.08025* (2026). doi: 10.48550/arXiv.2601.08025. url: https://arxiv.org/abs/2601.08025.

[22] Y. Matsubara, D. Callegaro, S. Singh, M. Levorato and F. Restuccia. "BottleFit: Learning Compressed Representations in Deep Neural Networks for Effective and Efficient Split Computing". In: *2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. IEEE, 2022, pp. 337–346. doi: 10.1109/WoWMoM54355.2022.00032.

[23] Y. Matsubara and M. Levorato. "Split Computing for Complex Object Detectors: Challenges and Preliminary Results". In: *Proceedings of the 4th International Workshop on Embedded and Mobile Deep Learning*. 2020. doi: 10.1145/3410338.3412338.

[24] Y. Matsubara, M. Levorato and F. Restuccia. "Split Computing and Early Exiting for Deep Learning Applications: Survey and Research Challenges". In: *ACM Computing Surveys* 55.5 (2022), pp. 1–30. doi: 10.1145/3527155.

[25] Microsoft. *Confidential VM Node Pools Support on AKS with AMD SEV-SNP Confidential VMs*. Microsoft Learn. Accessed: 2026-05-03. 2023. url: https://learn.microsoft.com/en-us/azure/confidential-computing/confidential-node-pool-aks.

[26] Microsoft. *Use Confidential Virtual Machines (CVM) in Azure Kubernetes Service (AKS)*. Microsoft Learn. Accessed: 2026-05-03. 2025. url: https://learn.microsoft.com/en-us/azure/aks/use-cvm.

[27] M. Misono, D. Stavrakakis, N. Santos and P. Bhatotia. "Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX". In: *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8.3 (Dec. 2024), pp. 1–42. doi: 10.1145/3700418.

[28] F. Mo, Z. Tarkhani and H. Haddadi. "Machine Learning with Confidential Computing: A Systematization of Knowledge". In: *ACM Computing Surveys* 56.11 (2024), pp. 1–40. doi: 10.1145/3670007.

[29] J. Na, H. Zhang, J. Lian and B. Zhang. "Partitioning DNNs for Optimizing Distributed Inference Performance on Cooperative Edge Devices: A Genetic Algorithm Approach". In: *Applied Sciences* (2022). doi: 10.3390/app122010619.

[30] A. Poudel, P. Niroula, C. MacDonald, L. Gloudemans and S. Herwig. "Mazu: A Zero Trust Architecture for Service Mesh Control Planes". In: *Proceedings of the 18th European Workshop on Systems Security (EuroSec '25)*. ACM, 2025. doi: 10.1145/3722041.3723100.

[31] P. Ren, X. Qiao, Y. Huang, L. Liu, C. Pu and S. Dustdar. "Fine-Grained Elastic Partitioning for Distributed DNN Towards Mobile Web AR Services in the 5G Era". In: *IEEE Transactions on Services Computing* 15.6 (2022), pp. 3260–3274. doi: 10.1109/TSC.2021.3098816.

[32] J. Shao and J. Zhang. "BottleNet++: An End-to-End Approach for Feature Compression in Device-Edge Co-Inference Systems". In: *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2020, pp. 1–6. doi: 10.1109/ICCWorkshops49005.2020.9145068.

[33] R. Stahl, A. Hoffman, D. Mueller-Gritschneder, A. Gerstlauer and U. Schlichtmann. "DeeperThings: Fully Distributed CNN Inference on Resource-Constrained Edge Devices". In: *International Journal of Parallel Programming* 49.4 (2021), pp. 600–624. doi: 10.1007/s10766-021-00712-3.

[34] Z. Taufique, A. Vyas, A. Miele, P. Liljeberg and A. Kanduri. "HiDP: Hierarchical DNN Partitioning for Distributed Inference on Heterogeneous Edge Platforms". In: *arXiv preprint arXiv:2411.16086* (2024). doi: 10.48550/arXiv.2411.16086. url: https://arxiv.org/abs/2411.16086.

[35] D. Xu, X. He, T. Su and Z. Wang. "A Survey on Deep Neural Network Partition over Cloud, Edge and End Devices". In: *arXiv preprint arXiv:2304.10020* (2023). doi: 10.48550/arXiv.2304.10020. url: https://arxiv.org/abs/2304.10020.

[36] M. Yan and K. Gopalan. "Performance Overheads of Confidential Virtual Machines". In: *2023 IEEE 31st International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2023. doi: 10.1109/MASCOTS59514.2023.10387607.

[37] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo and J. Zhang. "Edge Intelligence: Paving the Last Mile of Artificial Intelligence with Edge Computing". In: *Proceedings of the IEEE* 107.8 (2019), pp. 1738–1762. doi: 10.1109/JPROC.2019.2918951.

[38] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz et al. "Dissecting Overheads of Service Mesh Sidecars". In: *Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23)*. ACM, 2023, pp. 142–157. doi: 10.1145/3620678.3624652.

[39] X. Zhu, Y. Zhou, Y. Wang, X. Gao, A. Krishnamurthy, S. Kumar, R. Mahajan and D. Zhuo. "Rethinking RPC Communication for Microservices-based Applications". In: *Workshop on Hot Topics in Operating Systems (HotOS '25)*. ACM, 2025, pp. 158–164. doi: 10.1145/3713082.3730375.